

Einführung in den Compilerbau



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Author: Bassem Zouari

Version: v1.0

Semester: Wise 2023/2024

Stand: 14. Mai 2026

Fachbereich: Informatik

Inhaltsverzeichnis

1	Einleitung	3
1.1	Auswirkung der Zielmaschine	4
1.2	Aufbau von Compilern	7
1.3	Optimierung	10
1.4	Organisatorisches	11
1.5	Syntax	12
1.6	(Mini-) Triangle	14
1.7	Kontextuelle Einschränkungen	22
1.8	Semantik	24
2	Syntaktische Analyse	26
2.1	Kompilierung	26
2.2	Ablauf der Übersetzung	26
2.3	Ein-Pass Compiler	27
2.4	Multi-Pass Compiler	28
2.5	Vergleich Ein-Pass ./ Mulit-Pass-Compiler	28
2.6	Beispiel Multi-Pass	29
2.7	Aufbau des Triangle-Compilers	29
2.8	Syntaxanalyse	30
2.9	Subphasen der Syntaxanalyse	30
2.10	Token-Folge	31
2.11	Tokens	31
2.12	Aufzählung für Token-Arten (Auszug)	32
2.13	Parsen der Token-Folge	33
2.14	Aufbau des AST aus Token-Folge	33
2.15	Kurzwiederholung Grammatiken	34
2.16	Produktionen in EBNF	34
2.17	Reguläre Ausdrücke und Grammatiken	34
2.18	Transformation von Grammatiken	35
2.19	Grammatik-Transformation durch Gruppierung	35
2.20	Grammatik-Transformation durch Linksausklammern	35
2.21	Beseitigung von Linksrekursion	36
2.22	Kombinierter Fall	36
2.23	Ersetzen von Nicht-Terminalsymbolen	36
2.24	Nutzen von Grammatiktransformationen	37
2.25	Parsing Terminologie	37
2.26	Parsing Strategien	37
2.27	Beispiel: Micro-English	37



2.28	Bottom-Up Parsing	38
2.29	Beispiel Bottom-Up Parsing	38
2.30	Schwierigkeit bei Bottom-Up Parsing	40
2.31	Top-Down Parsing	40
2.32	Beispiel Top-Down Parsing	41
2.33	Hintergrund Top-Down Parsing	42
2.34	Vergleich Top-Down mit Bottom-Up Parsing	42
2.35	Parsen mit rekursivem Abstieg	43
2.36	Beispiel für Micro-English 1	43
2.37	Beispiel für Micro-English 2	43
2.38	Beispiel für Micro-English 3	44
2.39	Systematische Konstruktion von Parsern	45
3	Laufzeitorganisation	46
3.1	Einleitung	46
3.1.1	Wesentliche Aufgaben der Laufzeitorganisation	46
3.1.2	Zielmaschinensupport	47
3.1.3	Herausforderung des „Semantic Gap“	47
3.2	Triangle Abstract Machine	47
3.2.1	Speicherorganisation der TAM	47
3.2.2	TAM Speicherstruktur	48
3.3	Auswertung von Ausdrücken	48
3.3.1	Stack-Maschine	48
3.3.2	Register-Maschine	49

1 Einleitung

Compiler

Schnittstelle zwischen

- Programmiersprache
- Maschine

Programmiersprache Gut für Menschen handhabbar

- Smaltalk
- Java
- C++

Maschine Getrimmt auf

- Ausführungsgeschwindigkeit
- Preis/Chip-Fläche
- Energieverbrauch
- Nur selten: Leichte Programmierbarkeit

Auswirkung von Compilern

Entscheidet über dem Benutzer **zugängliche** Rechenleistung

Beispiel: Bildkompression auf Dothan CPU, 2 GHz

Compiler	Ausführungszeit	Programmgröße
GCC 3.3.6	7,5 ms	13 KB
ICC 9.0	6,5 ms	511 KB

Unterschiedliche Abstraktionsebenen

- Auf unteren Ebenen immer feinere Beschreibung
- Immer näher an Zielmaschine (Hardware)
- Details werden von Compiler hinzugefügt
 - Durch verschiedenste Algorithmen
 - * Analyse von Programmeigenschaften

Der Compiler untersucht den Code, um bestimmte Eigenschaften zu erkennen, z.B. welche Variablen oft verwendet werden oder welche Schleifen optimiert werden können.

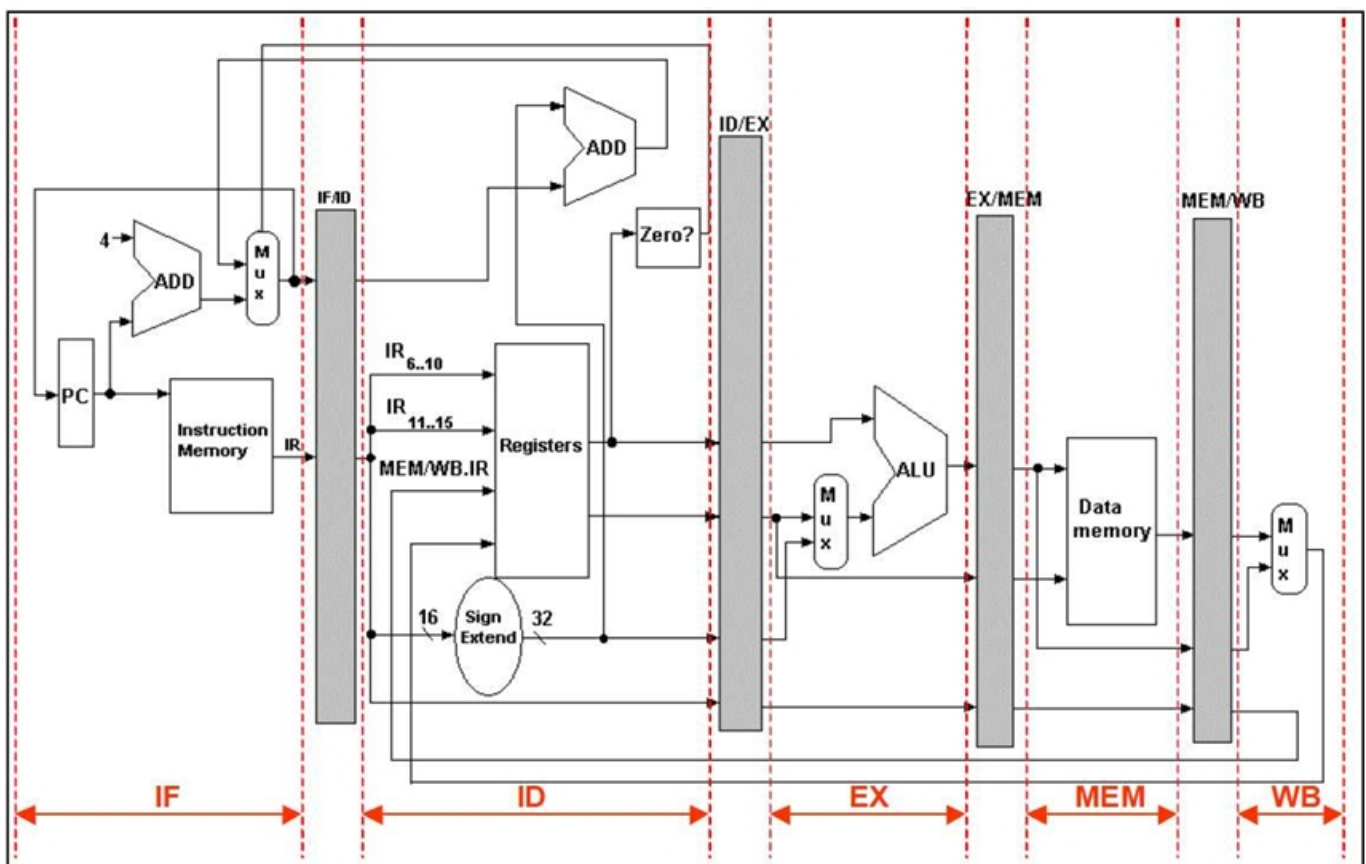
* Verfeinerung der Beschreibung durch Synthese von Details

Der Compiler fügt Details hinzu, die in der höheren Ebene nicht direkt angegeben wurden. z.B. kann er den Speicherbedarf optimieren oder bestimmte Maschinenbefehle einfügen.

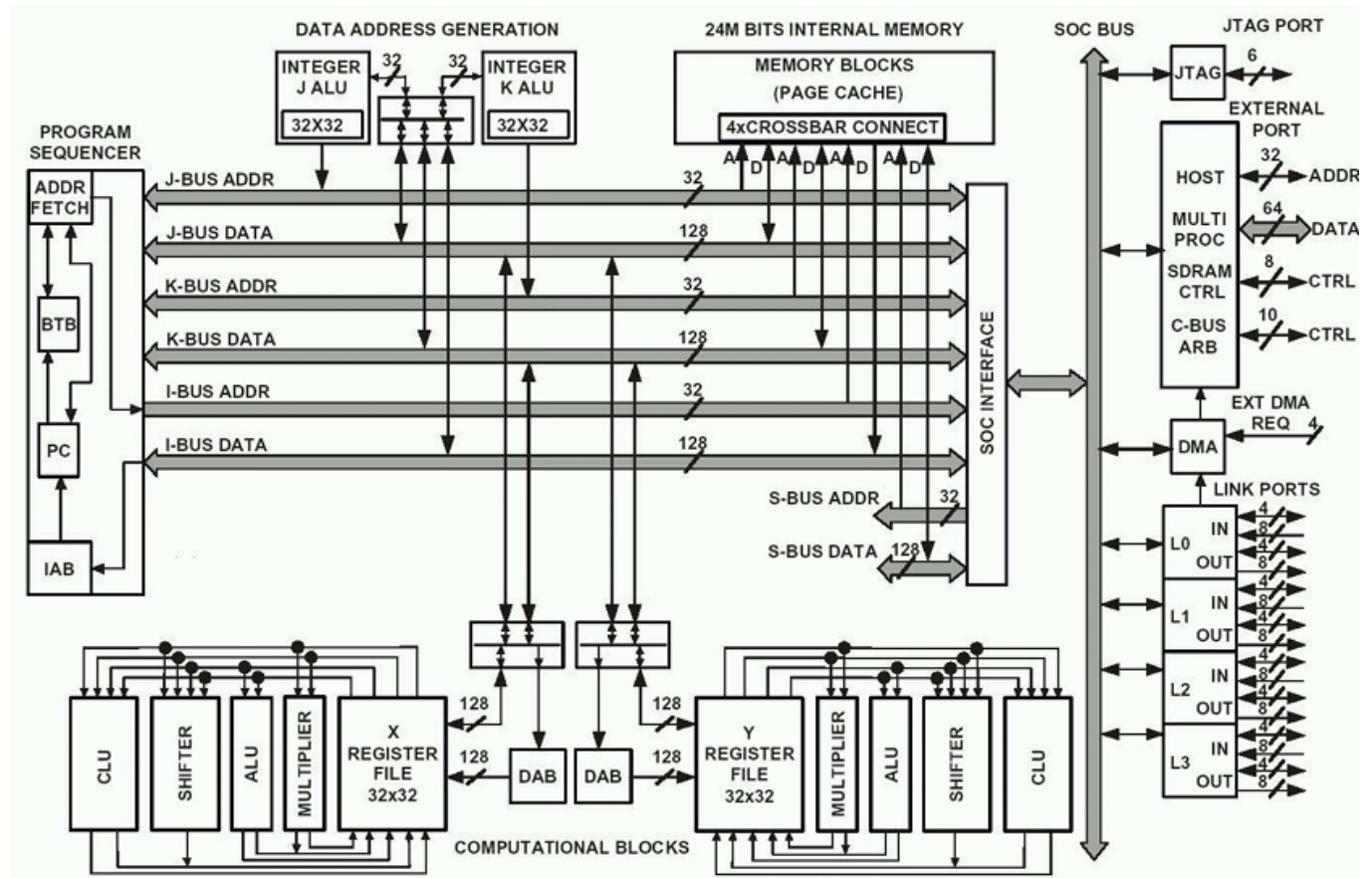
Zusammengefasst bedeutet das: Je näher man sich an der Hardware befindet, desto detaillierter wird der Code. Ein Compiler hilft dabei, die Lücke zwischen der abstrakten Programmierlogik und den maschinennahen Details zu schließen.

1.1 Auswirkung der Zielmaschine

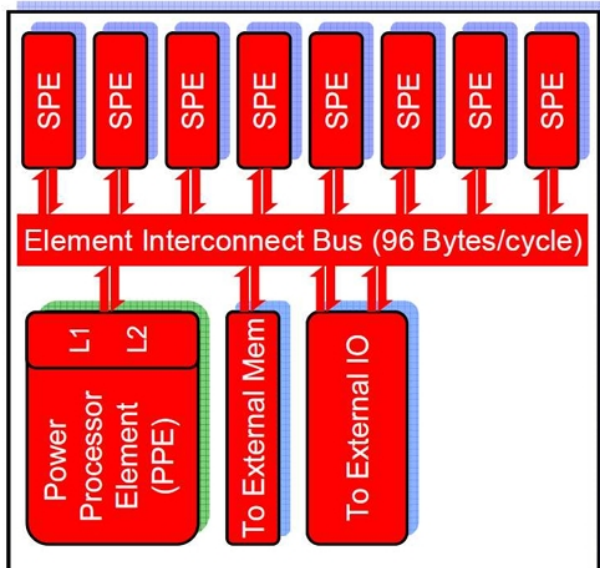
Einfach: Hennessy & Patterson DLX



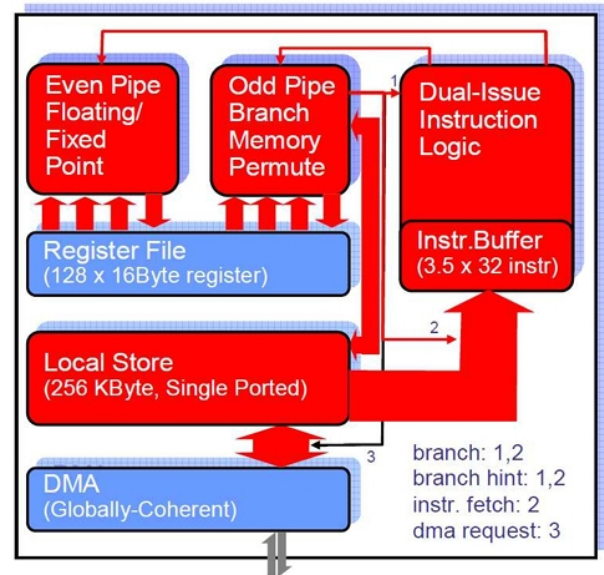
Komplizierter: Analog Devices TigerSHARC



Übersicht



SPE



Spezialisierte Anforderungen an CPUs

Je nach Anwendungsgebiet mehr oder weniger wichtig ...
Rechenleistung (hoch/niedrig)

- Datentypen (Gleitkomma, ganzzahlig, Vektoren)
- Operationen (Multiplikationen, MACs)
- Speicherbandbreiten (parallele Speicherzugriffe)
- Energieeffizienz
- Platzbedarf (für den Prozessorchip)

... können häufig nur durch spezialisierte Prozessoren erfüllt werden → **benötigen passende Compiler**

Technologie

- Taktfrequenz von Prozessoren nur mit Mühe steigerbar
- Trend weg von hochgetakteten Einzelprozessoren
- ... hin zu vielen (aber langsameren) Prozessoren
- Wie parallele Strukturen programmieren?
- Erste praktische Ansätze
 - OpenMP: Mehr-Kern-CPUs
 - NVidia CUDA: GPUs

- OpenCL: Heterogene Systeme (GPUs + CPUs, experimentell auch schon FPGAs)
- Aber noch wenig abstrakt
- Keine automatische Parallelisierung!

Extrem wichtiges Anwendungsgebiet: Ausführung von Machine Learning-Berechnungen (z.B. TensorFlow XLA, Glow, etc.)

Der Trend zu vielen langsameren Prozessoren ist ein Ansatz, um die Rechenleistung zu steigern, ohne dabei die Taktfrequenz zu erhöhen. Um das Potenzial solcher parallelen Architekturen zu nutzen, gibt es spezialisierte Programmierstechniken und Tools, wie OpenMP, CUDA und OpenCL. Für die Zukunft sind bessere, abstraktere Modelle nötig, die Entwicklern die Parallelisierung erleichtern und vielleicht irgendwann eine automatische Parallelisierung ermöglichen.

Motivation: Compilerbau in der Lehre

Kombination von verschiedenen Feldern der Informatik

- Theoretische Informatik (Parsing)
- Technische Informatik (Architektur der Zielmaschine)
- Praktische Informatik (Software-Engineering beim Aufbau des Compilers)

→ **Programmieraufgaben betreffen alle Felder!**

1.2 Aufbau von Compilern

Vorgehen: Bearbeitung in mehreren Phasen

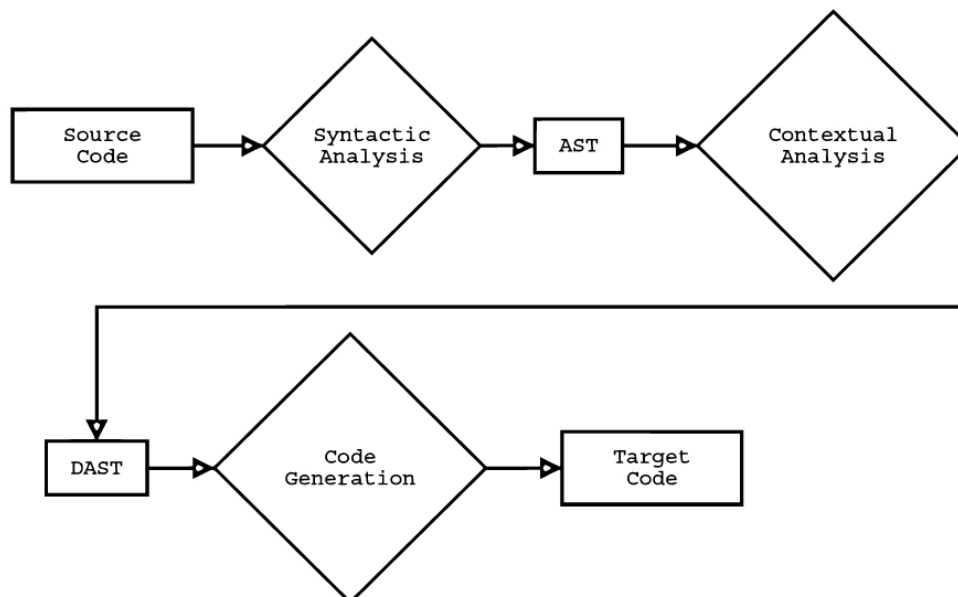
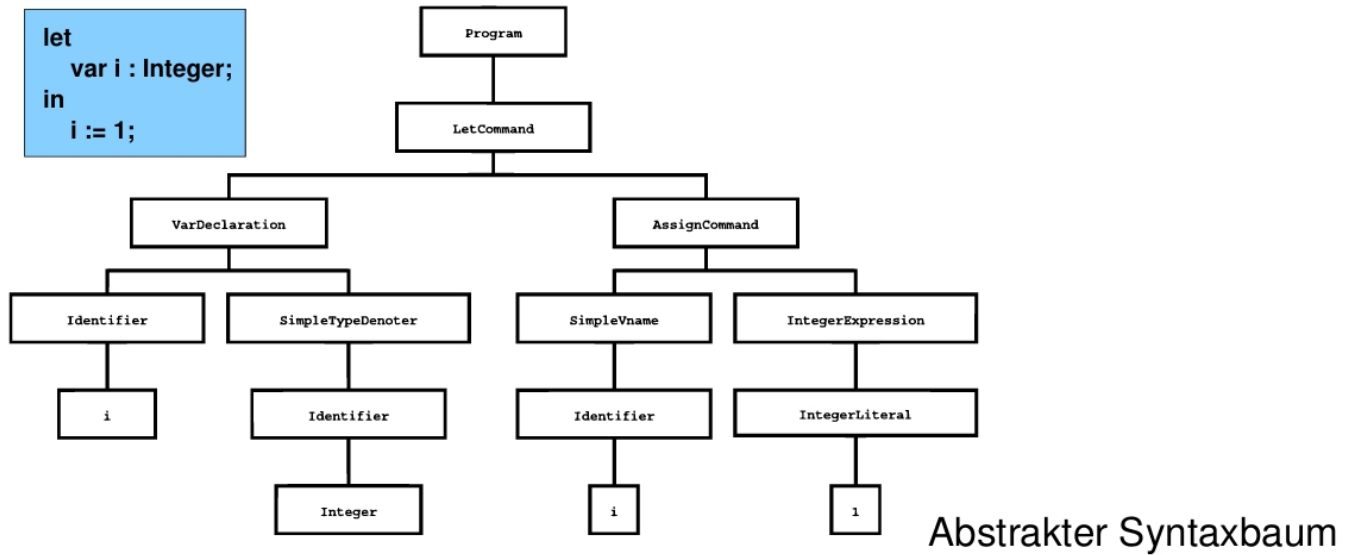


Abbildung 1: Zwischendarstellung(en) für den Informationsaustausch

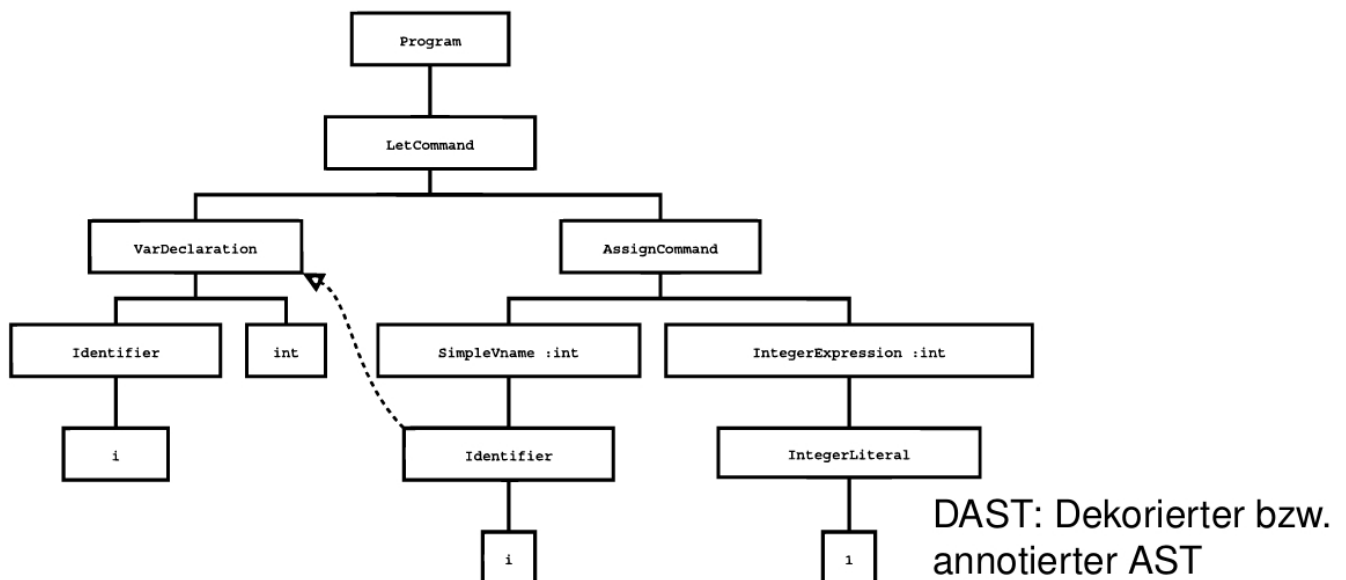
Syntaxanalyse

- Überprüfung ob Programm Syntaxregeln gehorcht
- Speichern des Programmes in geeigneter Darstellung



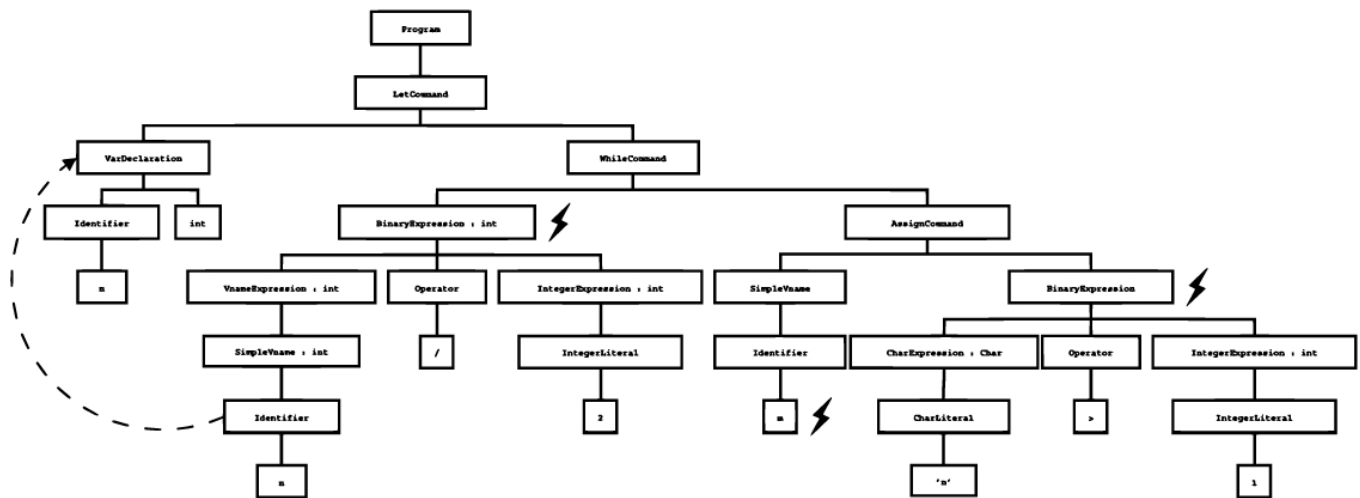
Kontextanalyse - Identifikation

- Ordne Variablen ihren Deklarationen zu
- Berechne Typen von Ausdrücken



Kontextanalyse - Überprüfung

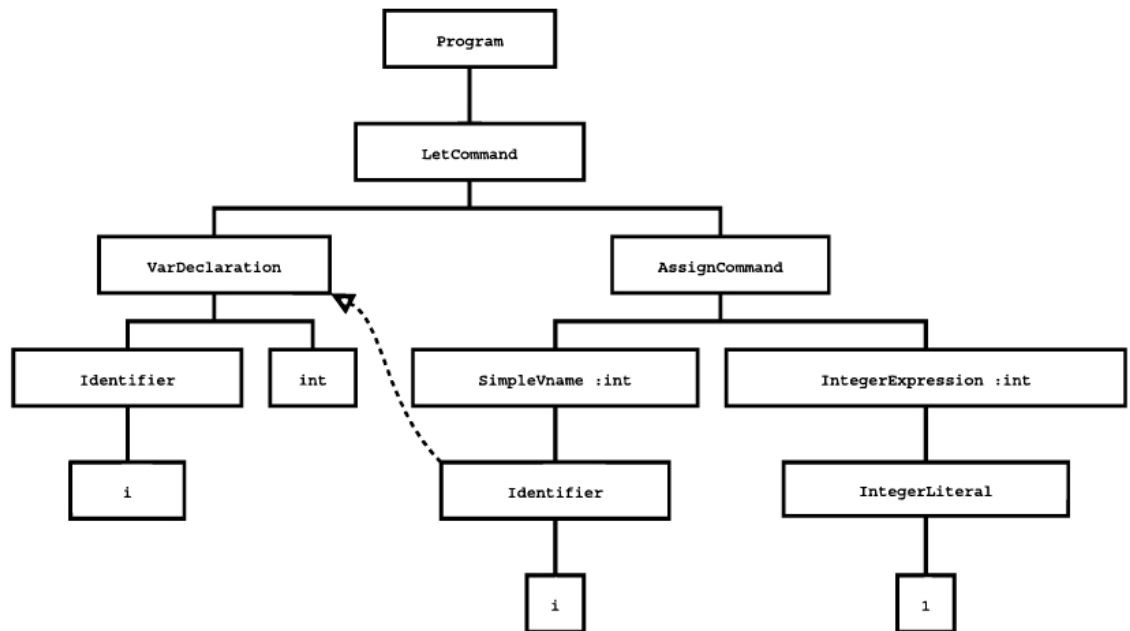
Erkenne Fehler in Variablen- und Typzuordnung



Code-Erzeugung 1

- Programm ist syntaktisch und kontextuell korrekt
- Übersetzung in Zielsprache
 - Maschinensprache
 - Assembler
 - C
 - Andere Hochsprache
- Ordne DAST-Teilen Instruktionen der Zielsprache zu
- Handhabung von Variablen
 1. Deklaration: Reserviere Speicherplatz für eine Variable
 2. Verwendung: Referenziere immer den zugeordneten Speicherplatz

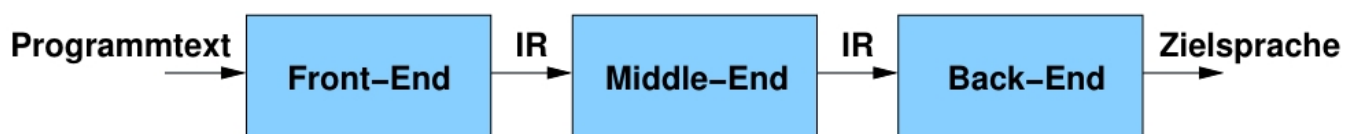
Code-Erzeugung 2



```
0:  PUSH      1          ; Platz fuer 'i' schaffen, Adresse 0[SB]
1:  LOADL     1          ; Wert 1 auf Stack legen
2:  STORE (1) 0[SB]      ; ein Datenwort vom Stack nach Adresse 0[SB] schreiben
3:  POP (0)   1          ; Platz von 'i' wieder aufgeben
4:  HALT
```

1.3 Optimierung

Optimierender Compiler



- Front-End: Syntaktische/kontextuelle Analyse
- Back-End: Code-Erzeugung
- **Middle-End:** Transformation von Zwischendarstellungen
 - Intermediate Representation (IR)
 - Keine direkte Code-Erzeugung aus Front-End IR
 - Verwendet in der Regel zusätzliche interne Darstellungen

Ziel: Verbesserung des erzeugten Codes in Bezug auf bestimmte Gütemaße

Beispiele für Optimierungen

Constant-Folding

```
1 x = (2+3) * y
```

```
1 x = 5*y
```

Common-Subexpression Elimination

```
1 x = 5 * a + b;  
2 y = 5 * a + c;
```

```
1 t = 5 * a;  
2 x = t + b;  
3 y = t + c;
```

Strength Reduction

```
1 for (i=0; i <= j; ++i){  
2     a[i*3] = 42;  
3 }
```

```
1 int t = 0;  
2 for (i=0; i <= j; ++i){  
3     a[t] = 42;  
4     t = t + 3;  
5 }
```

Loop-invariant Code Motion

```
1 int t;  
2 for (i=0; i <= j; ++i){  
3     t = x * y;  
4     a[i] = t * i;  
5 }
```

```
1 int t = x * y;  
2 for (i=0; i <= j; ++i){  
3     a[i] = t * i;  
4 }
```

1.4 Organisatorisches

Material Grundlage der Vorlesung

Grundlagen von Compilern: **Fast vollständig Programming Language Processors in Java** von David Watt und Deryck Brown, Prentice-Hall 2000 Auszugsweise noch weiteres Material, z.B. zum ANTLR Parsergenerator

Aufbau der Veranstaltung

Überblick über Front-End (ca. 3 Wochen)

- Lexing/Parsing
- Zwischendarstellungen

Überblick über Middle-End (ca. 2 Wochen)

- Semantische / Kontextanalyse

Überblick über Back-End (ca. 4 Wochen)

- Laufzeitorganisation
- Code-Erzeugung

Weiterführende Themen

- Verwendung von Front-End-Generatoren (ca. 2-3 Wochen)
- Java Virtuelle Maschine (ca. 1-2 Wochen)

1.5 Syntax

Syntax oder Grammatik (PLPJ, Kapitel 1)

Beschreibt die Satzstruktur von korrekten Programmen

- $n := n + 1;$
Syntaktisch korrektes Statement in Triangle
- „Ein Kreis hat zwei Ecken.“
Syntaktisch korrekte Aussage in Deutsch

Kontextuelle Einschränkungen

Dazu gehören Regeln für den Geltungsbereich (scope) und den Typ von Aussagen.

- n muss bei Auftreten des Statements passend deklariert sein.
- Kreise haben im allgemeinen keine Ecken.
Hier passen die Typen offenbar nicht.

Semantik

Die Bedeutung einer Anweisung/Aussage in einer Sprache. Wird bei Programmiersprachen häufig beschrieben ...
Operationell Welche Schritte laufen ab, wenn das Programm gestartet wird? **Denotational** Abbildung von Eingaben auf Ausgaben

Art der Spezifikation

- Für alle drei Teile
 1. Syntax
 2. Kontextuelle Einschränkungen
 3. Semantik
- ...gibt es jeweils zwei Spezifikationsarten
 - Formal
 - Informal

Triangle-Spezifikation

- Formale Syntax (reguläre Ausdrücke, EBNF)
- Informale kontextuelle Einschränkungen
- Informale Semantik

Syntax

Eine **Sprache** ist eine Menge von **Zeichenketten** aus einem **Alphabet**

- Wie diese Menge angeben?
- Bei endlichen Sprachen: Einfach Elemente aufzählen
- Geht nicht bei unendlichen Sprachen
- Einige mögliche Vorgehensweisen
 1. Mathematische Mengennotation
 2. Reguläre Ausdrücke
 3. Kontextfreie Grammatik

Syntax durch Mengenbeschreibung

Beispiele für die beschriebenen Zeichenketten

- $L = \{a,b,c\}$ beschreibt a, b, c
- $L = \{x^n | n > 0\}$ beschreibt x, xx, xxx, ...
- $L = \{x^n y^m | n > 0, m > 0\}$ beschreibt xxy, xyy, xxxyy, ...
- $L = \{x^n y^n | n > 0\}$ beschreibt xy, xxyy, ..., aber z.B. nicht xxy

Offensichtlich keine sonderliche nützliche und gut zu handhabende Spezifikationsform für komplexere Sprachen

Reguläre Ausdrücke (REs)

Erweitere Zeichenketten aus dem Alphabet um Operatoren

- **|** zeigt Alternativen an
- ***** zeigt Null oder mehr Vorkommen des vorangehenden Zeichens an
- ϵ ist die leere Zeichenkette
- **(...)** erlauben die Gruppierung von Teilausdrücken durch Klammerung

Beispiele

- $L = a|b|c$ ergibt a, b, c
- $L = ab^*$ ergibt a, ab, abb, ...
- $L = (ab)^*$ ergibt die leere Zeichenkette ϵ , ab, abab, ababab, ...
- $L = a(b|\epsilon)$ ergibt a oder ab

Mächtigkeit von REs?

Kann man die Menge aller regulären Ausdrücke selber durch einen regulären Ausdruck beschreiben?

Nein! REs sind daher ungeeignet zur Beschreibung der Syntax komplexer Programmiersprachen

...also Weitersuchen nach geeigneter Beschreibungsform für Syntax

Aber: REs sind trotzdem innerhalb eines Compilers nützlich (siehe PLPJ Kapitel 4, Scanner)

Kontextfreie Grammatiken (CFGs)

Eine kontextfreie Grammatik besteht aus

- Einer Menge von Terminalsymbolen T aus Alphabet
- Einer Menge von Nicht-Terminalsymbolen N
- Einem Startsymbol $S \in N$
- Einer Menge von Produktionen P
 - Beschreiben, wie Nicht-Terminalsymbole aus Terminalsymbolen zusammengesetzt sind

Produktionen in Backus-Naur-Form (BNF) Nicht-Terminal ::= **Zeichenkette** aus Terminal und Nicht-Terminalsymbolen

Produktionen in Extended BNF (EBNF) Nicht-Terminal ::= **RE** aus Terminal und Nicht-Terminalsymbolen

Mehrdeutigkeit in BNF

Gegeben seien die Produktionen:

$$\begin{aligned} S &::= S+S \\ S &::= x \end{aligned}$$

Wie lässt sich die Zeichenkette $x+x+x$ herleiten?

$$S \rightarrow S+S \rightarrow x+S \rightarrow x+S+S \rightarrow x+x+S \rightarrow x+x+x$$

Aber auch anders:

$$S \rightarrow S+S \rightarrow S+x \rightarrow S+S+x \rightarrow S+x+x \rightarrow x+x+x$$

Eindeutige CFG

Für sinnvolle praktische Anwendungen müssen CFGs **eindeutig** sein.
Eindeutige Produktionen für die gleiche CFG:

$$\begin{aligned} S &::= x + S \\ S &::= x \end{aligned}$$

1.6 (Mini-) Triangle

(Mini-) Triangle

- Pascal-artige Sprache als Anschauungsobjekt
- Compiler-Quellcode auf Web-Page
- In der Vorlesung: Mini-Triangle
 - Weiter vereinfachte Version
 - z.B. keine Definition von Unterfunktionen

```

let
  const MAX ~ 10;
  var n: Integer
in begin
  getint(var n);
  if (n>0) /\ (n<=MAX) then
    while n > 0 do begin
      putint(n); puteol();
      n := n-1
    end
  else
  end
end

```

Lokale Deklarationen

Konstante (hässliches "~"!)

Variable kann in `getint` verändert werden

Folge von Anweisungen zwischen `begin/end`

`else` ist erforderlich (darf aber leer sein)

Produktionen der Mini-Triangle CFG 1

```

Program ::= single-Command
single-Command ::= empty
                | V-name := Expression
                | Identifier ( Expression )
                | if Expression then single-Command
                  else single-Command
                | while Expression do single-Command
                | let Declaration in single-Command
                | begin Command end
Command ::= single-Command
          | Command ; single-Command
...

```

Produktionen der Mini-Triangle CFG 2

```
Expression
  ::= primary-Expression
     | Expression Operator primary-Expression
primary-Expression
  ::= Integer-Literal
     | V-name
     | Operator primary-Expression
     | ( Expression )
V-name ::= Identifier
Identifier ::= Letter
           | Identifier Letter
           | Identifier Digit
Integer-Literal ::= Digit
                | Integer-Literal Digit
Operator ::= + | - | * | / | < | > | =
```

Produktionen der Mini-Triangle CFG 3

```
Declaration
  ::= single-Declaration
     | Declaration ; single-Declaration
single-Declaration
  ::= const Identifier ~ Expression
     | var Identifier : Type-denoter
Type-denoter ::= Identifier
```

Produktionen der Mini-Triangle CFG 4

```
Comment ::= ! CommentLine eol
CommentLine ::= Graphic CommentLine
Graphic ::= any printable character or space
Digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Terminologie

Phrase Von einem gegebenen Nicht-Terminalsymbol herleitbare Kette von Terminalsymbolen. z.B. Expression-Phrase, Command-Phrase ... **Satz** S-Phrase, wobei S das Startsymbol der CFG ist Beispiel:

```
1 let
2     var y : Integer
3 in
4     y := y + 1
```

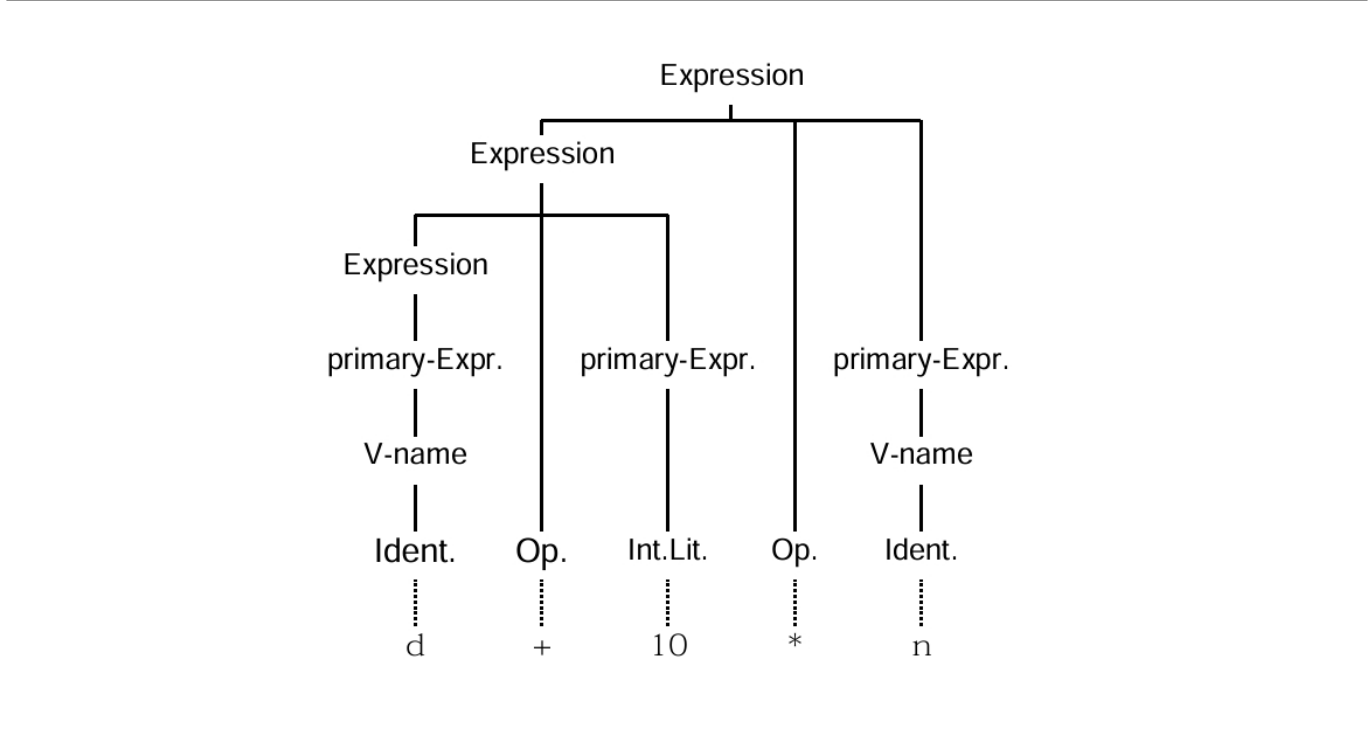
- Das gesamte Programm ist ein Satz der CFG
- Zeile 2 ist eine single-Declaration-Phrase
- Zeile 4 ist eine single-Command-Phrase

Syntaxbäume

Ein Syntaxbaum ist ein geordneter, markierter Baum bei dem

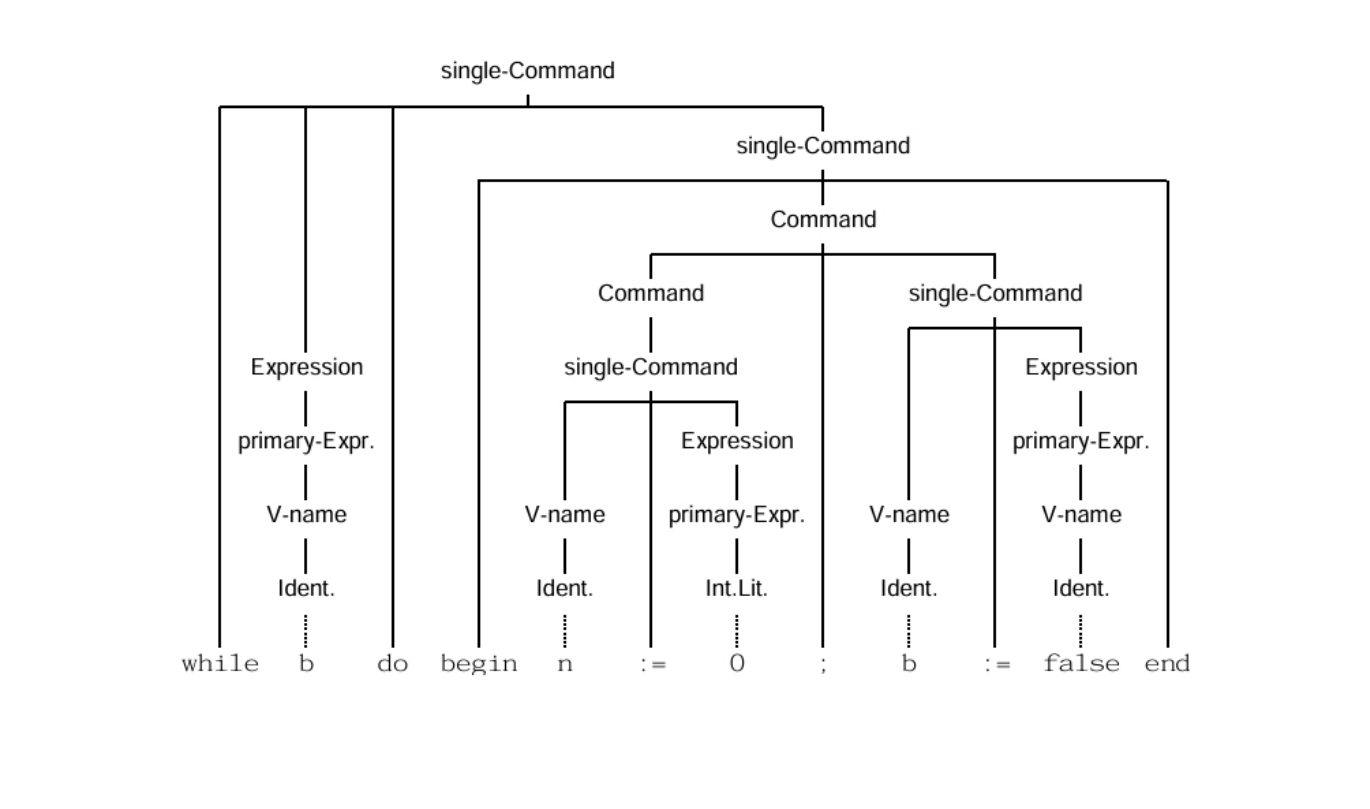
- ...die Blätter mit Terminalsymbolen markiert sind
- ...die inneren Knoten mit Nicht-Terminalsymbolen markiert sind
- ...jeder innere Knoten N (von links nach rechts) die Kinder $X_1, \dots, X_n$ hat, entsprechend der Produktion $N := X_1 \dots X_n$

Ein N-Baum ist ein Baum mit einem N Nicht-Terminalsymbol am Wurzelknoten.



single-Command-Baum

```
1 while b do begin
2     n := 0;
3     b := false
4 end
```



Konkrete und abstrakte Syntax

- Grammatik spezifiziert präzise syntaktische Details
 - do, :=, ...

→ Konkrete Syntax, wichtig für das Verfassen korrekter Programme

- Konkrete Syntax hat aber keinen Einfluss auf Semantik der Programme
 - $V = E$
 - $V := E$
 - set $V = E$
 - assign E to V
 - $V \leftarrow E$
- ...können alle das gleiche bedeuten: Eine Zuweisung von E nach V

→ Für weitere Verarbeitung Darstellung vereinfachen!

Abstrakte Syntax 1

- Modelliert nur **essentielle Information**
- Idee: Orientierung an der Subphrasen-Struktur der Produktionen
- Beispiel: $v\text{-name} := \text{Expression}$ hat zwei Subphrasen
 1. $V\text{-name}$
 2. Expression

Abstrakte Syntax 2

- Schlüsselworte, Begrenzer wie **do**, **:=** sind irrelevant
- Unterscheidung zwischen
 - **Command** und **single-Command**
 - **Declaration** und **single-Declaration**
 - **Expression** und **primary-Expression**
- ... sind nur für das Erkennen des Programmes relevant, nicht zur Darstellung seiner Semantik.

→ Alle dafür unwichtigen Details weglassen!

Auszug aus der abstrakten Syntax 1

Command

<code>::= V-name := Expression</code>	<i>AssignCmd</i>
<code> Identifier (Expression)</code>	<i>CallCmd</i>
<code> if Expression then Command</code>	
<code> else Command</code>	<i>IfCmd</i>
<code> while Expression do Command</code>	<i>WhileCmd</i>
<code> let Declaration in Command</code>	<i>LetCmd</i>
<code> Command ; Command</code>	<i>SequentialCmd</i>

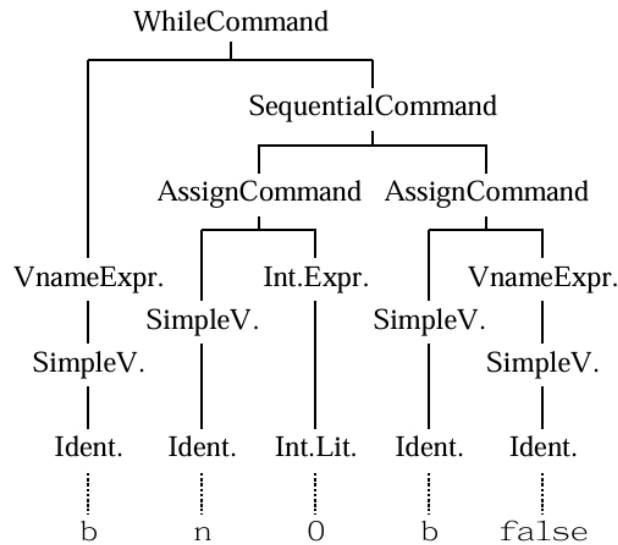
Auszug aus der abstrakten Syntax 2

Expression

<code>::= Integer-Literal</code>	<i>IntegerExp</i>
<code> V-name</code>	<i>VnameExp</i>
<code> Operator Expression</code>	<i>UnaryExp</i>
<code> Expression Op Expression</code>	<i>BinaryExp</i>
<code>V-name ::= Identifier</code>	<i>SimpleVName</i>

Beispiel abstrakte Syntax

```
while b do begin
  n := 0;
  b := false
end
```



AST als Zwischendarstellung 1

- AST ist eine weit verbreitete Form der IR
- High-level IR
- Sehr nah an der Eingabesprache
- Gut geeignet für weitreichende Analysen und Transformationen
 - Unabhängig von Architektur der Zielmaschine
 - Verschieben von Anweisungen
 - Änderungen der Programmstruktur

Schlechter geeignet für maschinennahe Analysen und Transformationen

- Ausnutzung von Maschinenregistern
- Ausnutzung von speziellen Maschinenbefehlsfolgen

→ Hier: Konzentration auf **maschinenunabhängige** Ebene

- (D)AST ist Hauptrepräsentation
- Für einzelne Bearbeitungsschritte: Andere IRs

1.7 Kontextuelle Einschränkungen

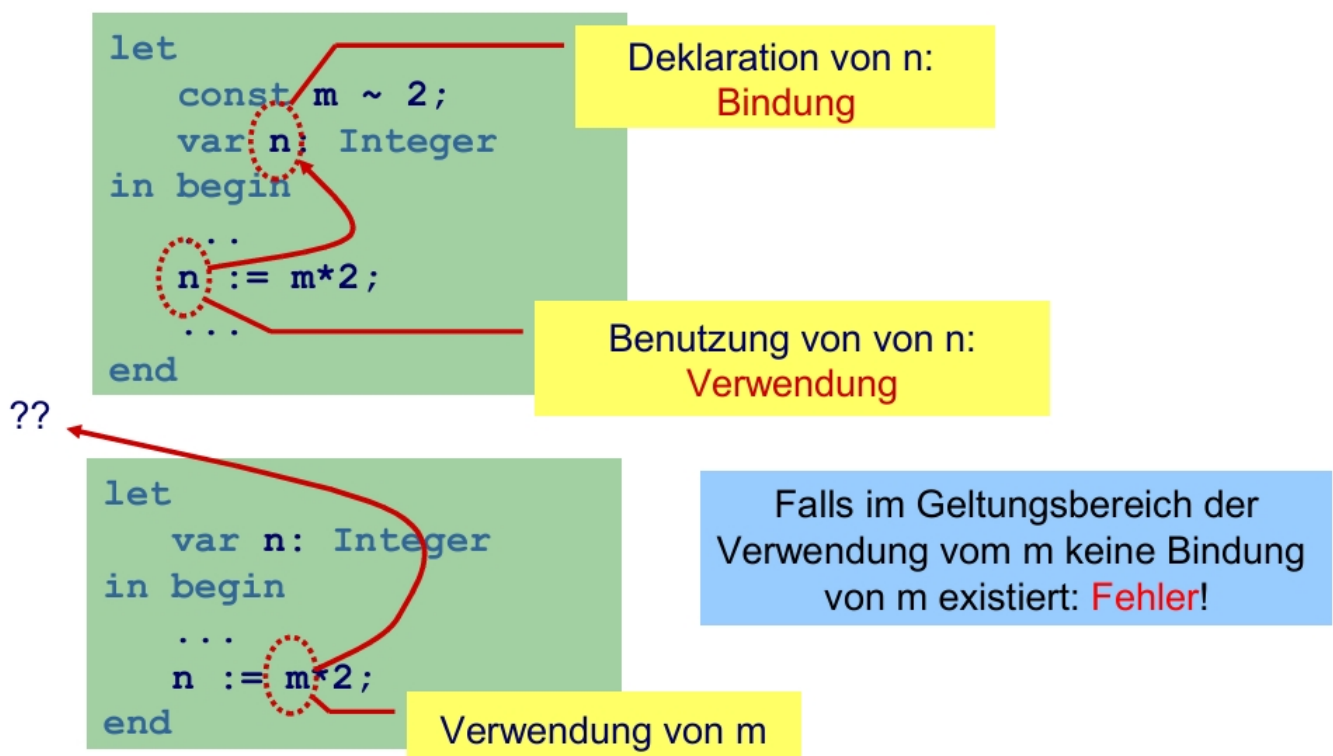
Kontextuelle Einschränkungen: Geltungsbereiche

Syntaktische Korrektheit reicht nicht aus für sinnvolle Übersetzung

Geltungsbereiche(Scope)

- Betreffen Sichtbarkeit von Bezeichnern
- Jeder verwendete Bezeichner muss vorher deklariert werden
 - ... nicht bei allen Programmiersprachen
- Deklaration ist sog. bindendes Auftreten des Bezeichners
- Benutzung ist sog. verwendendes Auftreten des Bezeichners
- Aufgabe: Bringe jede Verwendung mit genau der einen passenden Bindung in Zusammenhang

Beispiele Geltungsbereichen



Kontextuelle Einschränkungen: Typen

- Jeder Wert hat einen Typ
- Jede Operation
 - ...hat Anforderungen an die Typen der Operanden
 - ...hat Regeln für den Typ des Ergebnisses

...auch nicht bei allen Programmiersprachen

- Hier: statische Typisierung (zur Compile-Zeit)
- Alternativ: dynamische Typisierung (zur Laufzeit)

Beispiele Typen

```
let
  var n: Integer
in begin
  ...
  while n > 0 do
    n := n-1;
  ...
end
```

Typregel für $E_1 > E_2$ (GreaterOp):
Wenn E_1 und E_2 beide vom Typ **int**,
dann ist **Ergebnis** vom Typ **bool**.

Typregel für **while** E **do** C (WhileCmd):
 E muss vom Typ **boolean** sein.

Typregel für $V := E$ (AssignCmd):
Die Typen von V und E müssen
äquivalent sein.

Typregel für $E_1 - E_2$ (SubOp):
Wenn E_1 und E_2 beide vom Typ **int**,
dann ist **Ergebnis** vom Typ **int**.

1.8 Semantik

Semantik

Semantik beschreibt die Bedeutung eines Programmes zur Ausführungszeit.
Allgemeine Terminologie:

...werden **ausgeführt**. Mögliche Seiteneffekte:

- Ändern der Werte von Variablen
- Ein-/Ausgabeoperationen

Ausdrücke

...werden **ausgewertet**(evaluiert), um ein Ergebnis zu erhalten.

- Die Evaluation kann in einigen Sprachen auch Seiteneffekte haben.

Deklarationen

...werden **elaboriert** um eine Bindung vorzunehmen. Mögliche Seiteneffekte:

- Allokieren von Speicherplatz
- Initialisieren von Speicherplatz

Beispiele Semantik von (Mini)Triangle

Die Beschreibung orientiert sich am AST.

AssignCmd $v := E$

1. Der Ausdruck E wird evaluiert um einen Wert V zu erhalten
2. V wird an die Variable v zugewiesen

BinaryExp $E_1 op E_2$

1. Der Ausdruck E_1 wird evaluiert um einen Wert V_1 zu erhalten
2. Der Ausdruck E_2 wird evaluiert um einen Wert V_2 zu erhalten
3. Die Werte V_1 und V_2 werden mit dem Operator op zu einem Wert V_3 verknüpft
4. V_3 ist das Ergebnis der BinaryExp

Declaration $\text{var } I : T$

1. Der Bezeichner I wird an eine Variable vom Typ T gebunden
2. Es wird ein für T passender Speicherbereich bereitgestellt
3. Der Speicherbereich ist nicht initialisiert
4. Der Geltungsbereich für I ist der eingeschlossenen Block (LetCmd)
5. Am Ende des Blockes wird die Bindung aufgehoben
6. ...und der Speicherbereich wieder freigegeben

Triangle

- In Vorlesung: Mini-Triangle
 - Stark vereinfacht
 - z.B. Keine Unterprogramme (Prozeduren/Funktionen)
- Im praktischen Teil: Triangle
 - Pascal-artige Sprache
 - Arrays, Records, Prozeduren, Funktionen
 - Parameterübergabe durch Wert oder Referenz
 - Prozeduren/Funktionen als Parameter erlaubt
 - Ausdrücke haben keine Seiteneffekte
- Beschreibung in PLPJ, Anhang B

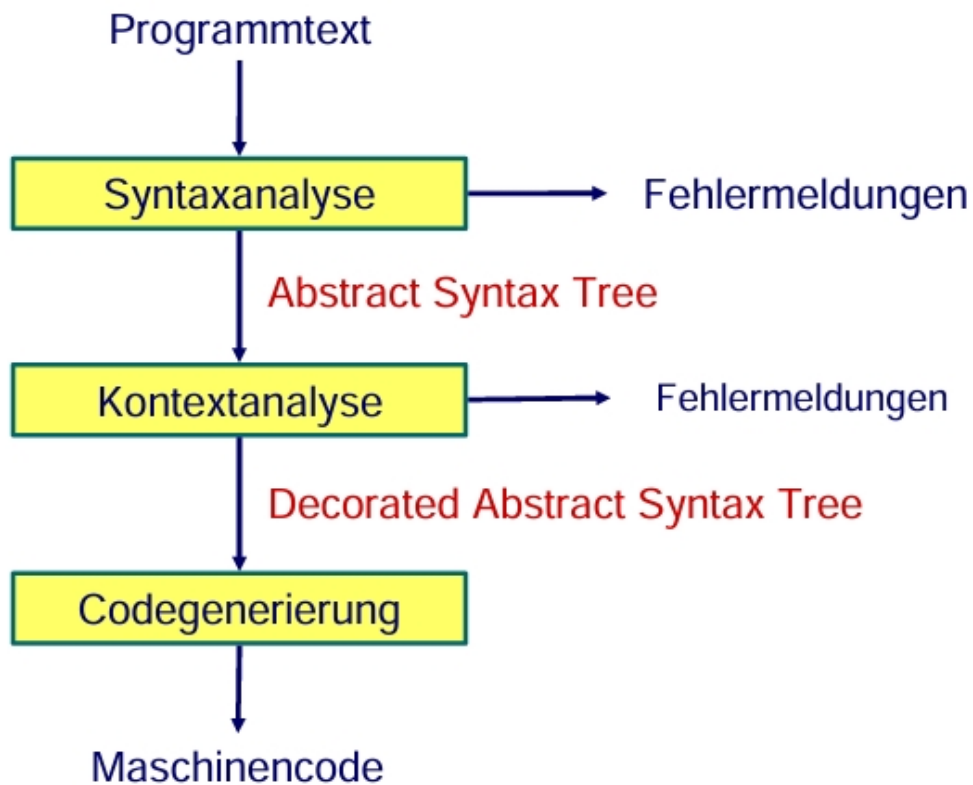
2 Syntaktische Analyse

2.1 Kompilierung

2.2 Ablauf der Übersetzung

Terminologie: Phase

- Transformationsschritte
 - Von Quellcode
 - ...zum Maschinencode
- Entspricht häufig den Teilen der Sprachspezifikation
 1. Syntax → Syntaxanalyse
 2. Kontextuelle Einschränkungen → Kontextanalyse
 3. Semantik → Codegenerierung



Terminologie: Durchgang (pass)

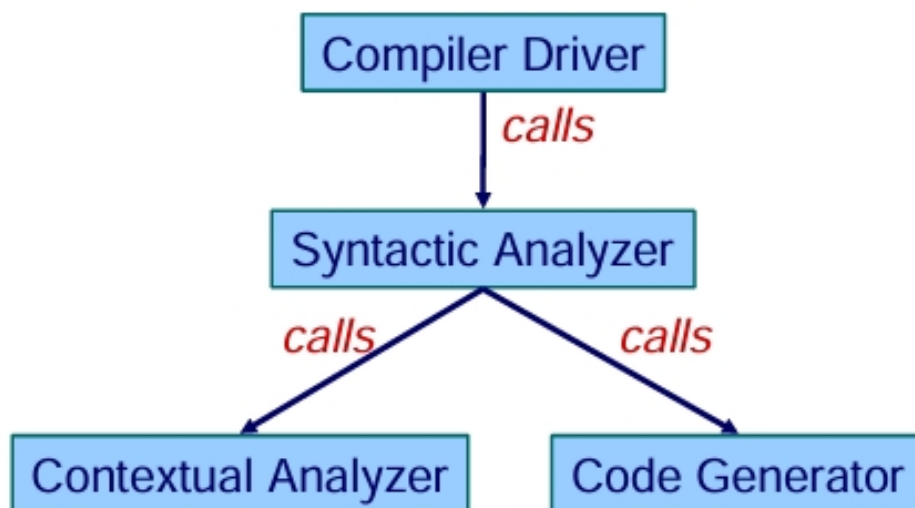
- Kompletter Durchgang des Programmes
- Läuft über Quelltext oder IR
- Pass kann Phase entsprechen
- ...muss aber nicht!
- Einzelner Pass kann mehrere Phasen durchführen

- Aufbau des Compilers wird von der Anzahl der Passes dominiert

Ein Durchgang (Pass) ist ein vollständiger Ablauf, bei dem der Compiler den Code einmal analysiert oder bearbeitet. Ein Compiler kann aus einem oder mehreren Passes bestehen. Mehrere Passes erlauben tiefere Optimierungen und bessere Fehleranalyse, was die Effizienz des erzeugten Codes verbessern kann, aber den Kompilierungsprozess verlängert. Die Anzahl der Passes beeinflusst daher wesentlich den Aufbau und die Leistung des Compilers.

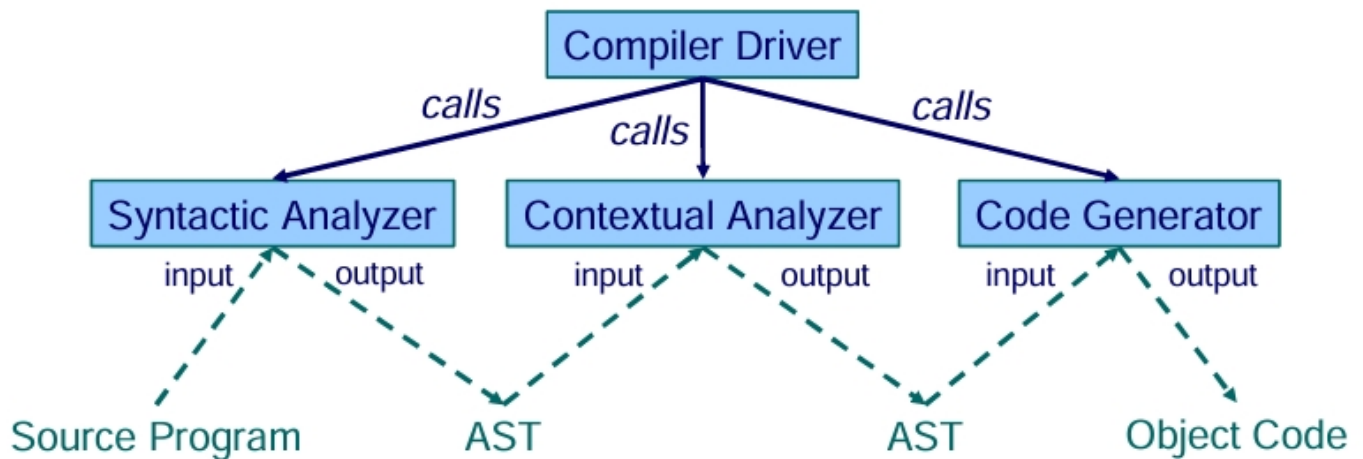
2.3 Ein-Pass Compiler

- Macht nur **einen** Pass über den Quelltext
 - Baut in der Regel **keine** echte IR auf
- Führt gleichzeitig aus
 - Syntaxanalyse (Parsing)
 - Kontextanalyse
 - Codegenerierung
- Pascal Compiler haben häufig Ein-Pass-Struktur



2.4 Multi-Pass Compiler

- Macht mehrere Passes über das Program
 - Quelltext und IR
- Datenweitergabe zwischen Passes über IR



2.5 Vergleich Ein-Pass ./ Mulit-Pass-Compiler

	Ein-Pass	Multi-Pass
Laufzeit	+	-
Speicher	+ für große Prog.	+ für kleine Prog.
Modularität	-	+
Flexibilität	-	+
Globale Optim.	--	+
Eingabesprachen	Nicht für alle	

Müssen Bezeichner vor Verwendung
deklariert werden?

2.6 Beispiel Multi-Pass

Java-Compilierung **erfordert** mehrere Passes

```
1 class Example{
2     void inc() {n = n + 1;}
3     int n;
4     void use() {n = 0; inc();}
5 }
```

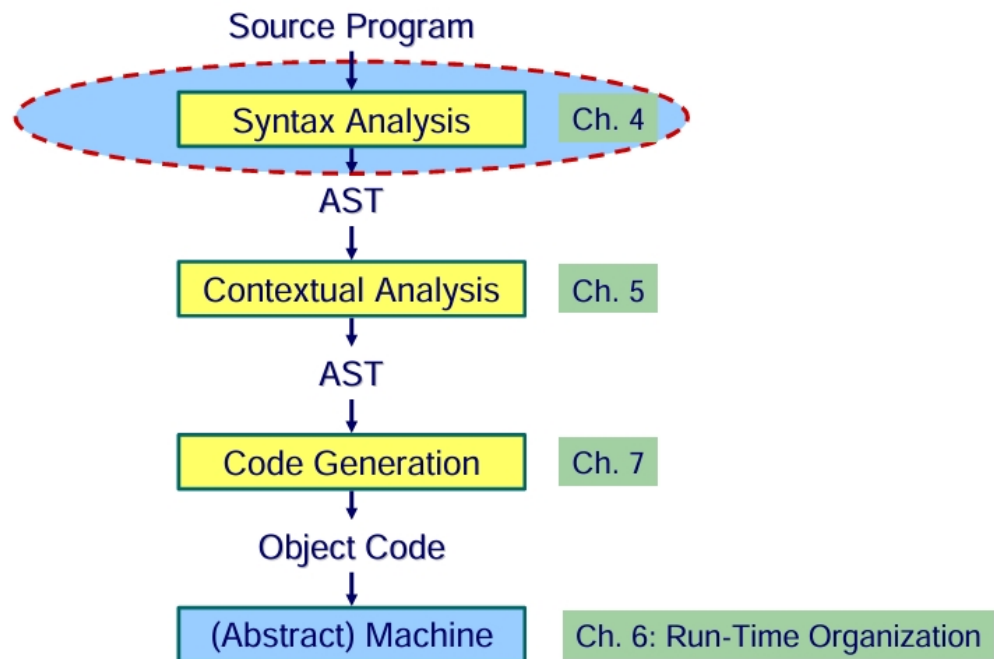
Beachte Reihenfolge Verwendung/Bindung von n!

2.7 Aufbau des Triangle-Compilers

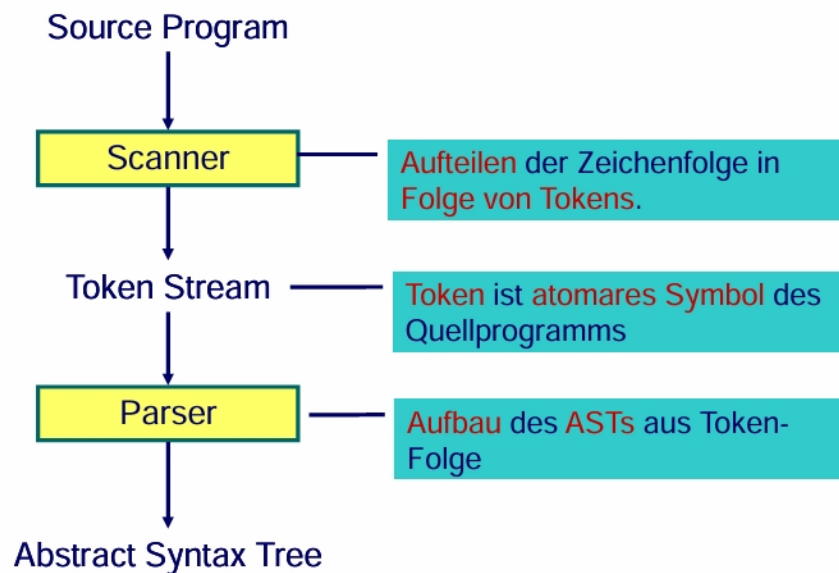
- Ein-Pass wäre für Triangle möglich
- Aus pädagogische Gründen aber Multi-Pass

```
1 public class Compiler{
2     public static boolean compileProgram(...){
3         Scanner scanner = new Scanner(...);
4         Parser parser = new Parser(...);
5         Checker checker = new Checker(...);
6         Encoder encoder = new Encoder(...);
7         ...
8         Program theAST = parser.parse();
9         checker.check(theAST);
10        encoder.encode(theAST);
11        ...
12    }
13    public static void main (String[] args){
14        ... compileProgram(...); ...
15    }
16 }
```

2.8 Syntaxanalyse



2.9 Subphasen der Syntaxanalyse



2.10 Token-Folge

Beispielprogramm in Triangle

```
! Groesster Gemeinsamer Teiler
let func gcd(x: Integer, y: Integer) : Integer ~
    if x // y = 0                ! // -> Modulo
    then y
    else gcd(y, x // y);
in  putint(gcd(321,81))
```

Token-Folge: Ohne Leerzeichen, Zeilenvorschub und Kommentare

```
let func gcd ( x : Integer , y : Integer )
: Integer ~ Integer if x // y = 0 then y
else gcd ( y , x // y ) ; in putint ( gcd
( 321 , 81 ) )
```

2.11 Tokens

- **Token** ist atomares Symbol des Programms
- Verwendet zwischen Scanner und Parser
- Kann auch aus mehreren Zeichen bestehen
- Zeichen selbst i.d.R. uninteressant, Ausnahmen:
 - Bezeichnernamen
 - Konstante Werte (Zahlen, Zeichen), sog. Literale
- ...Parser ist nur an der **Art** des Tokens interessiert

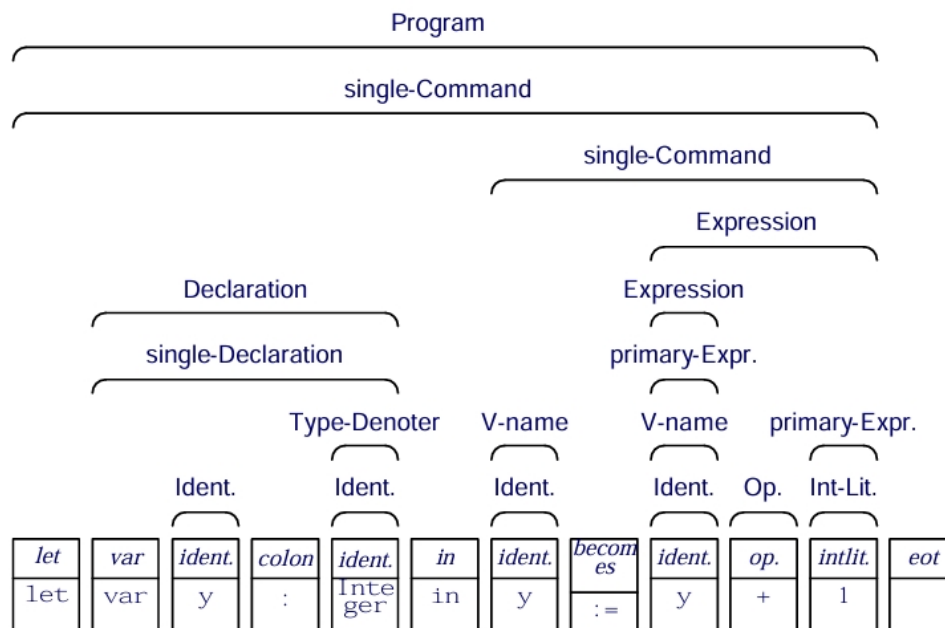
```
1 class Token {
2     TokenKind kind; //enum TokenKind...
3     String spelling;
4     SourcePosition position; //Zeilennummer, Spalte
5 }
```

2.12 Aufzählung für Token-Arten (Auszug)

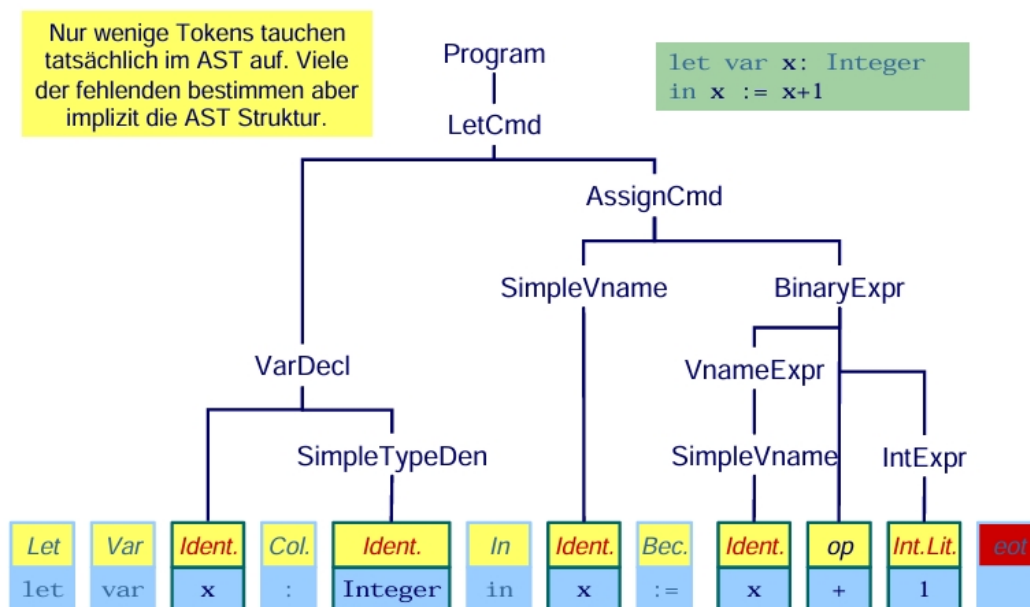
```
1  enum TokenKind{
2      //literals, identifiers, operators ...
3      INTLITERAL (0, "<int>"),
4      IDENTIFIER (2, "<identifier>"),
5      OPERATOR   (3, "<operator>"),
6
7      //reserved words
8      BEGIN      (5, "begin"),
9      CONST      (6, "const"),
10     DO          (7, "do"),
11     ELSE        (8, "else"),
12     END         (9, "end"),
13     IF          (11, "if"),
14     IN          (12, "in"),
15     LET         (13, "let"),
16     THEN        (17, "then"),
17     VAR         (19, "var"),
18     WHILE       (20, "while"),
19     SEMICOLON   (23, ";"),
20     COMMA       (24, ","),
21     BECOMES     (25, "!="),
22     IS          (26, "~"),
23     LPARSEN     (27, "("),
24     RPARSEN     (28, ")"),
25     ...
26     final int id;
27     final String spelling;
28     ...
29 }
```

Beispiel: `t = new Token()TokenKind.OPERATOR, "+", position);`

2.13 Parsen der Token-Folge



2.14 Aufbau des AST aus Token-Folge



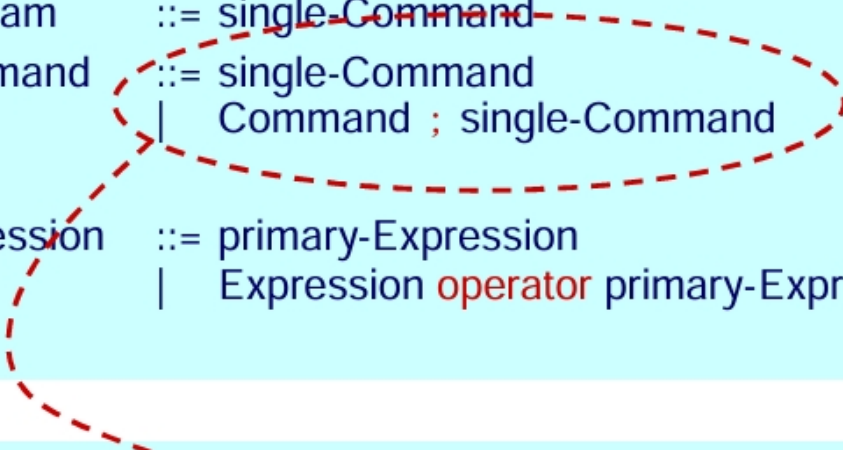
2.15 Kurzwiederholung Grammatiken

- Kontextfreie Grammatiken (CFG)
- Spezifiziert durch (N, T, P, S)
- CFG definiert Menge von Zeichenketten
 - Elemente sind Sätze bestehend aus Terminalsymbolen
 - Gesamtmenge ist Sprache der CFG
- Hier: Sätze haben eindeutige Phrasenstruktur
- P häufig in Backus-Naur-Form (BNF) angegeben
- Übersichtlicher: Extended BNF
 - BNF + Reguläre Ausdrücke auf rechter Seite der Produktionen

2.16 Produktionen in EBNF

BNF

```
Program ::= single-Command
Command ::= single-Command
          | Command ; single-Command
....
Expression ::= primary-Expression
            | Expression operator primary-Expression
```



EBNF

```
Command ::= single-Command ( ; single-Command ) *
....
Expression ::= primary-Expression
              ( operator primary-Expression ) *
```

2.17 Reguläre Ausdrücke und Grammatiken

- Auch REs definieren eine Sprache
 - **Reguläre Sprache**
 - Weniger komplex als durch CFG beschreibbare Sprachen
- CFG erlaubt Beschreibung von Selbsteinbettung

- Ausdruck $a*(b+c)/d$ bettet Ausdruck $b+c$ ein
- Vergleichbar dem Konzept der Rekursion
- REs erlauben **keine** Beschreibung von Selbsteinbettung

Ziel: Systematische Herleitung von Parsern aus CFG

2.18 Transformation von Grammatiken

Hilfsmittel

- CFG kann transformiert (umgestellt) werden
- ...unter Beibehaltung der beschriebenen Sprache

2.19 Grammatik-Transformation durch Gruppierung

- Zusammenfassen von Produktionen mit gleichem Nicht-Terminal auf linker Seite
 - Left-Hand Side (LHS), analog RHS

Vor Transformation

```
S ::= X + S
S ::= X S ::= ε
```

Nach Gruppierung

```
S ::= X + S | X | ε
```

2.20 Grammatik-Transformation durch Linksausklammern

- Zusammenfassen von gleichen Anfängen in einer Produktion
- $XY \mid XZ \rightarrow X(Y \mid Z)$

Beispiel:



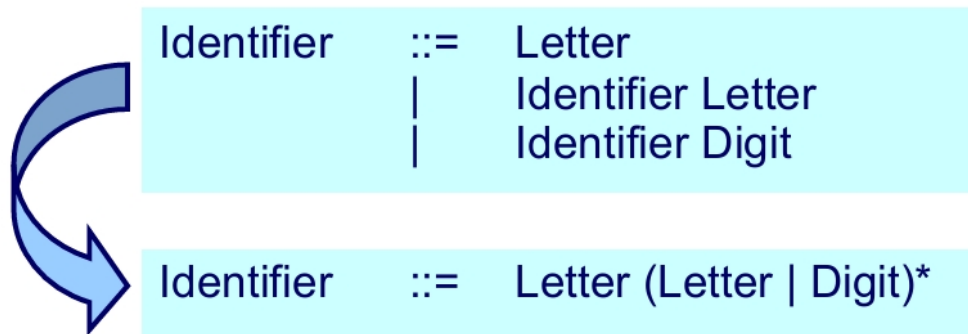
```
cmd ::= if Expr then cmd
      | if Expr then cmd else cmd

cmd ::= if Expr then cmd (ε | else cmd)
```

2.21 Beseitigung von Linksrekursion

- Linksrekursion in Produktion
 - $N ::= X \mid N Y$
 - $L(N) = \{X, XY, XYY, XYYY, XYYYY, \dots\}$
- Ersetzung durch
 - $N ::= X(Y)^*$

Beispiel:



2.22 Kombierter Fall

Vor Transformation

$N ::= X_1 \mid \dots \mid X_m \mid N Y_1 \mid N Y_n$

Nach Linksausklammern

$N ::= (X_1 \mid \dots \mid X_m) (N (Y_1 \mid \dots \mid Y_n))$

Nach Beseitigen der Linksrekursion

$N ::= (X_1 \mid \dots \mid X_m) (Y_1 \mid \dots \mid Y_n)^*$

2.23 Ersetzen von Nicht-Terminalsymbolen

- Wenn $N ::= X$ einzige Produktion mit LHS N ist
- $\dots N$ durch X in RHS aller Produktionen ersetzen

Beispiel:

Vor Transformation

$\text{single-Declaration} ::= \text{var Identifier : Type-denoter} \mid \dots$
 $\text{Type-denoter} ::= \text{Identifier}$

Nach Ersetzung

$\text{single-Declaration} ::= \text{var Identifier : Identifier} \mid \dots$

Aber ...

Solche „überflüssigen“ Nicht-Terminals können nützlichen Dokumentationscharakter für den menschlichen Leser haben!

2.24 Nutzen von Grammatiktransformationen

- Hier auf den ersten Blick noch nicht erkennbar
- Erlauben kompaktere und lesbarere Beschreibung von CFGs
- **Sehr nützlich** bei der Konstruktion von Parsern für CFGs

2.25 Parsing Terminologie

Erkennung: Entscheidung, ob ein Eingabetext ein Satz der Grammatik G ist.

Parsing: Erkennung und zusätzlich Bestimmung der Phrasen-Struktur

- Beispiel: Durch konkreten Syntaxbaum

Eindeutigkeit: Eine Grammatik ist eindeutig, falls jeder Eingabetext auf maximal eine Weise geparsed werden kann

- Ein syntaktisch korrekter Eingabetext hat genau einen eindeutigen Syntaxbaum

2.26 Parsing Strategien

- Zwei wesentliche Verfahren
- Unterscheiden sich in der Art ihres Vorgehens
 - **Top-Down** Beispiel: Rekursiver Abstieg
 - **Bottom-Up** Beispiel: Shift/Reduce

2.27 Beispiel: Micro-English

Produktionen

Sentence ::= Subject Verb Object
Subject ::= I | a Noun | the Noun
Object ::= me | a Noun | the Noun
Noun ::= cat | mat | rat Verb ::= like | is | see | sees

Beispiel der erzeugten Sprache

the cat sees a rat.
I like the cat.
the cat see me.
I like me.
a rat like me.

2.28 Bottom-Up Parsing

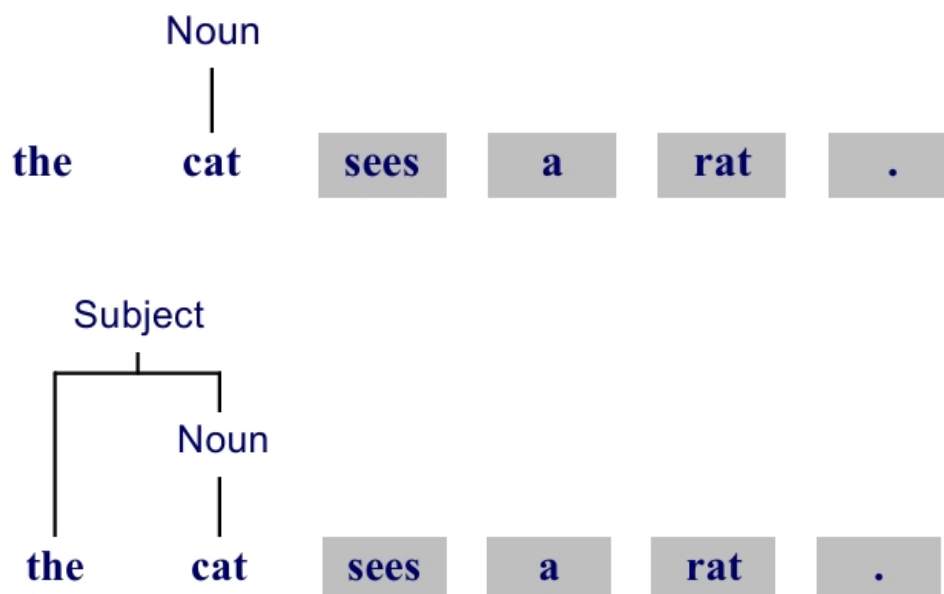
Vorgehensweise

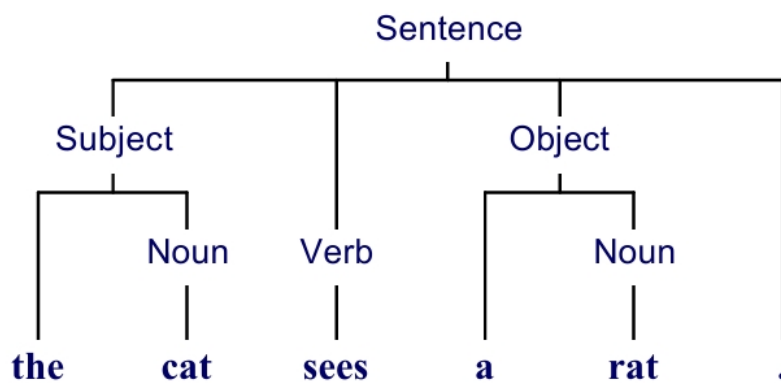
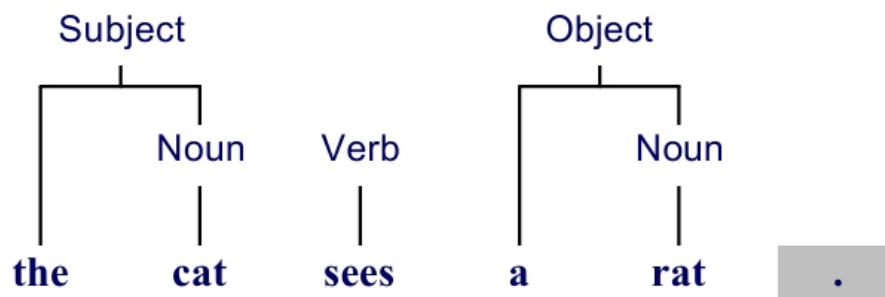
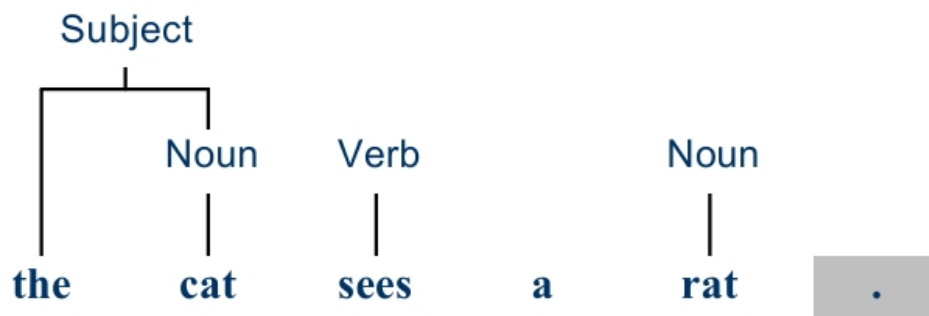
- Untersuche Eingabetext zeichenweise, von links nach rechts
- Baue Syntaxbaum vom **unten nach oben** auf
 - Von den Terminalzeichen in den Blättern
 - ...zum S Nicht-Terminal in der Wurzel

Zwei Arten von Aktionen

- **Shift** Lese Zeichen ein
 - Zusätzlich: Und lege es auf dem Stack ab
- **Reduce** Erkenne ein Nicht-Terminal LHS der Produktion p
 - Zusätzlich: Oberste Elemente des Stapels müssen RHS von p entsprechen, ersetze durch LHS von p (Zusammenfassen)
 - Ende wenn Startsymbol S erreicht und Eingabetext komplett gelesen

2.29 Beispiel Bottom-Up Parsing





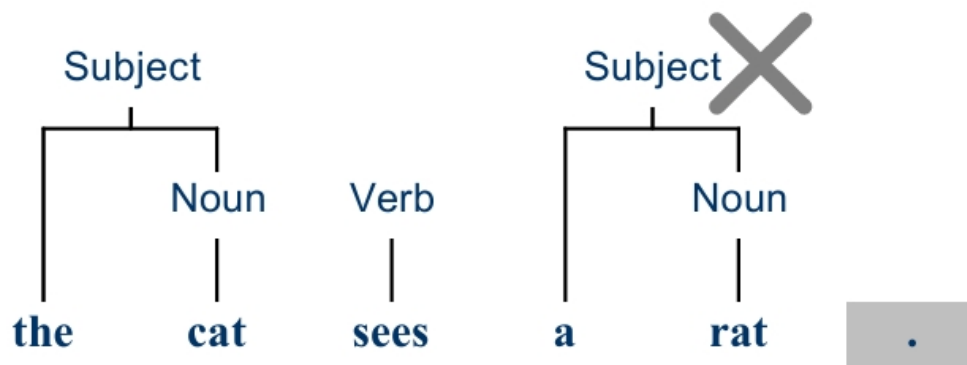
Sentence ::= Subject Verb Object .
 Subject ::= I | a Noun | the Noun
 Object ::= me | a Noun | the Noun
 Noun ::= cat | mat | rat
 Verb ::= like | is | see | sees

Parsing-Schritte (Bottom-Up - Shift-Reduce)

1. **Shift** the: Füge the in den Stack ein.
2. **Shift** cat: Füge cat in den Stack ein.
 - Jetzt haben wir the cat im Stack.
3. **Reduce** the cat: Ersetze the cat durch **Subject**.
 - Regel: **Subject ::= the Noun**
 - Jetzt steht Subject im Stack.
4. ...

2.30 Schwierigkeit bei Bottom-Up Parsing

Welche Produktion beim Zusammenfassen anwenden? Lösung: Nicht nur bekannte Zeichen betrachten, sondern auch



noch Zustand („schon Subject gesehen“) einbeziehen.

... aber hier nicht weiter vertieft!

Beim **Bottom-Up Parsing** besteht die Schwierigkeit darin, zu entscheiden, welche Regel angewendet werden soll, um Symbole zusammenzufassen. Der Parser muss nicht nur die aktuelle Symbole betrachten, sondern auch seinen **Parsing-Zustand** mit einbeziehen (z.B. ob er bereits ein Subjekt erkannt hat), um Entscheidungen zu treffen und den Satz korrekt zu analysieren.

2.31 Top-Down Parsing

Vorgehensweise

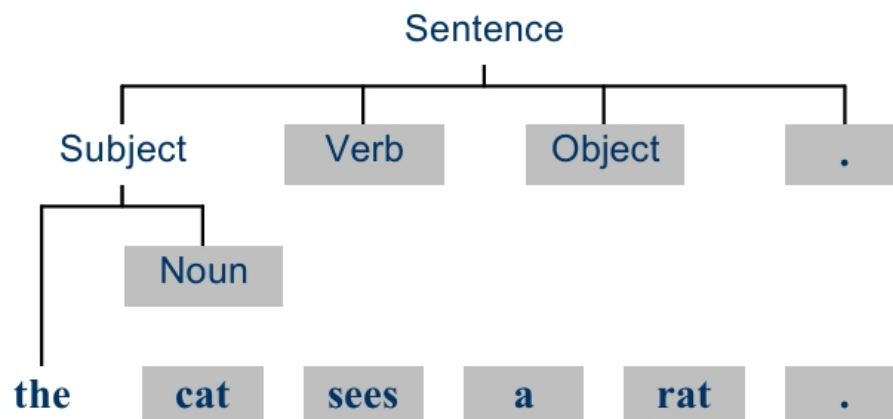
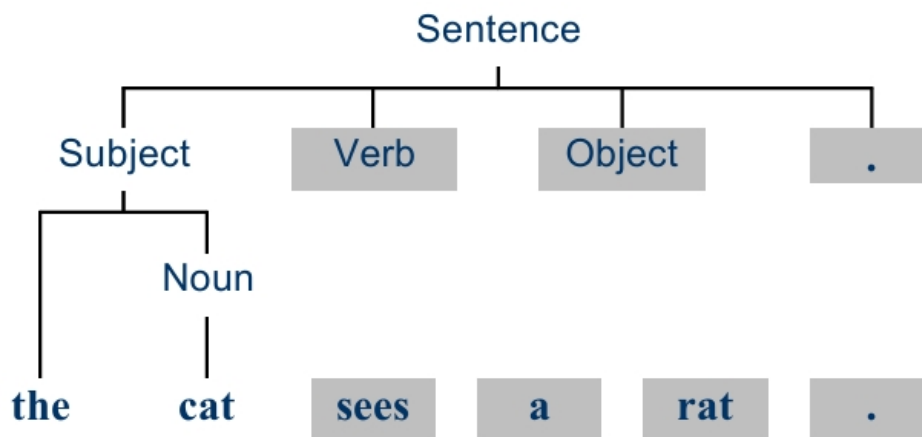
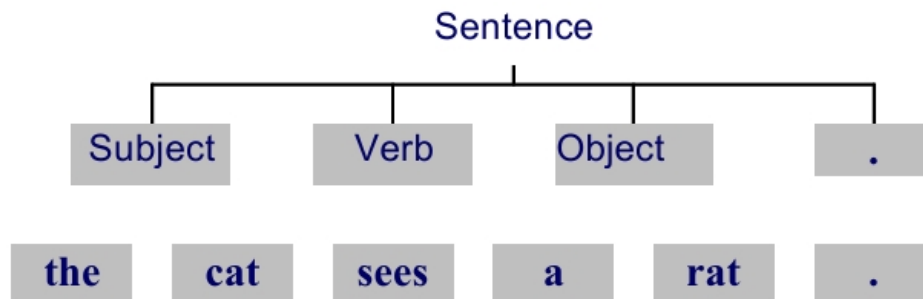
- Untersuche Eingabetext zeichenweise, von links nach rechts
- Baue Syntaxbaum von **oben nach unten** auf
 - Vom Start-Nicht-Terminal S in der Wurzel
 - ...zu den Terminalzeichen in den Blättern

Aktion

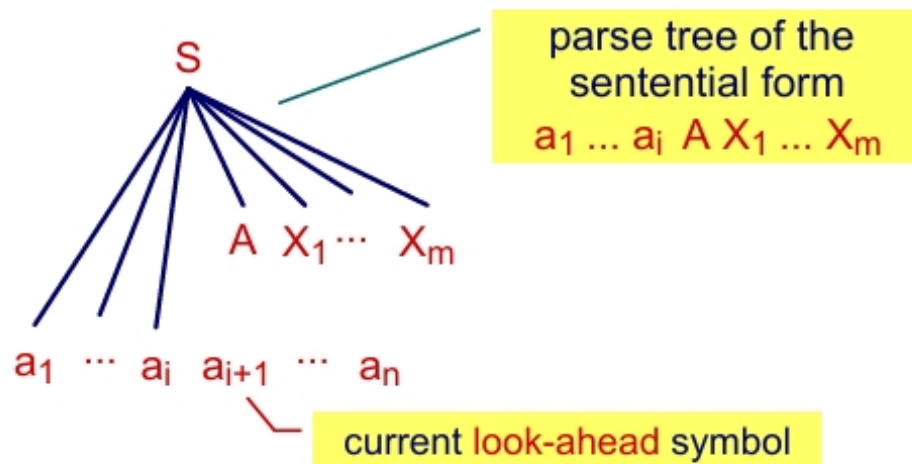
- Expandiere jeweils das am weitesten links gelegene Nicht-Terminal N
- ...durch Anwendung einer Produktion **N ::= X**
- Wähle Produktion aus durch Betrachten der nächsten n Zeichen des Eingabetextes (Annahme hier: n = 1)

- Falls keine Produktion auf Zeichen passt → **Fehler!**
- Ende wenn Eingabetext komplett gelesen und kein unexpandiertes Nicht-Terminal mehr existiert

2.32 Beispiel Top-Down Parsing



2.33 Hintergrund Top-Down Parsing



Falls es möglich ist,

- ...bei Betrachten der nächsten k Zeichen des Textes
- ...immer die richtige Produktion zu finden

dann ist die Grammatik **LL(k)**

- **L**: Lese Eingabetext von **links nach rechts**
- **L**: Leite immer vom am weitesten **links** stehenden Nicht-Terminal ab

2.34 Vergleich Top-Down mit Bottom-Up Parsing

- Probleme mit Top-Down-Parsing
 - Konstruktion einer LL(k) Grammatik für die gewünschte Sprache gelegentlich mühsam
 - Linksausklammern und Beseitigen von Linksrekursion können Lesbarkeit der Grammatik erschweren
- Lösung: Bottom-Up-Parsing mit LR(k)-Techniken
 - **L**: Lese Eingabetext von **links nach rechts**
 - **R**: Fasse die am weitesten **rechts** stehenden Terminal-Symbole zusammen und baue den Baum **rückwärts** auf
 - Mächtigeres Beschreibungsinstrument als LL(k)
 - Nachteil: Parsing-Vorgang komplexer und schlechter verständlich

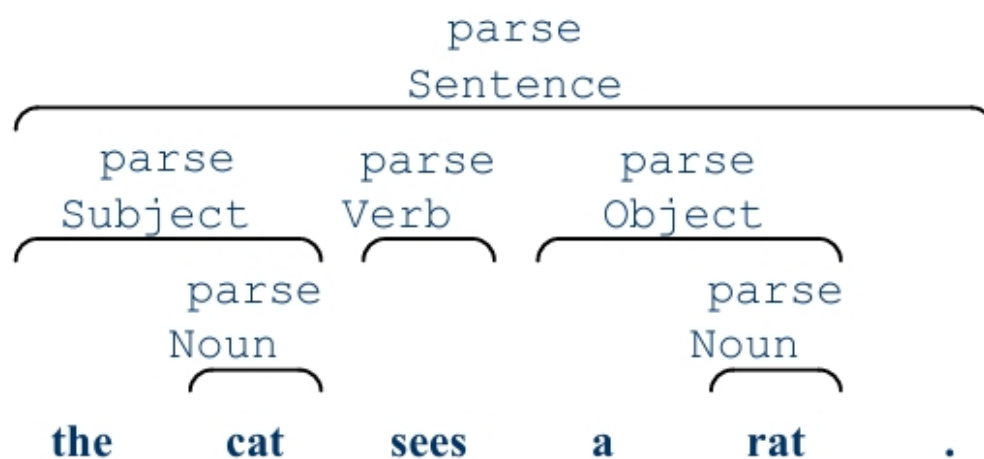
LL(k) $\subset$ LR(k)

2.35 Parsen mit rekursivem Abstieg

Einfache Implementierung der Top-Down Strategie, Idee:

- Struktur des konkreten Syntaxbaumes (Parse-Baum) entspricht
- ...Aufrufmuster von sich wechselseitig aufrufenden Prozeduren
- Für jedes Nicht-Terminal **XYZ** existiert
- ...Parse-Prozedur **parseXYZ**, die genau dieses Nicht-Terminal parst

Beispiel:



2.36 Beispiel für Micro-English 1

Sentence ::= Subject Verb Object .

```
1 protected void parseSentence(){
2     parseSubject();
3     parseVerb();
4     parseObject();
5     accept(".");
6 }
```

accept(t) prüft, ob **aktuelles Token** das erwartete Token **t** ist.

2.37 Beispiel für Micro-English 2

Subject ::= **I** | **a** Noun | **the** Noun

```

1  protected void parseSubject(){
2      if (currentToken matches "I"){
3          accept("I");
4      } else if (currentToken matches "a"){
5          accept("a");
6          parseNoun();
7      } else if (currentToken matches "the"){
8          accept("the");
9          parseNoun();
10     } else
11         report a syntax error
12 }

```

Die Methode **muss** immer anhand von `currentToken` die **passende** Alternative auswählen können.

2.38 Beispiel für Micro-English 3

```

public class MicroEnglishParser {
    protected Token currentToken;

    public void parse() {
        currentToken = first token;
        parseSentence();
        check that no token follows the sentence
    }

    protected void accept(Token expected) { ... }
    protected void parseSentence() { ... }
    protected void parseSubject() { ... }
    protected void parseObject() { ... }
    protected void parseNoun() { ... }
    protected ...
}

```

Schnittstelle zum Scanner, der die Tokens liefert

In Watt & Brown sind die Parse-Methoden als **private** deklariert. Ungeschickt, da es die Anpassung des Verhaltens durch Vererbung verhindert.

- **currentToken** enthält nacheinander die Tokens des Eingabetextes
- Ablauf einer Methode **parseN**
 - Bei Eintritt enthält **currentToken** eines der Tokens, **mit denen N beginnen kann**
 - ...sonst wäre eine andere Parse-Methode aufgerufen werden (oder Syntaxfehler)
 - Bei Austritt enthält **currentToken** das auf die **N**-Phrase folgende Token
- Ablauf der Methode **accept(t)**
 - Bei Eintritt muss **currentToken = t** sein
 - ...sonst Syntaxfehler
 - Bei Austritt enthält **currentToken** das auf t folgende Token

2.39 Systematische Konstruktion von Parsern

Entwicklung von Parsern mit rekursivem Abstieg

1. Formuliere Grammatik (CFG) in EBNF
 - Eine Produktion pro Nicht-Terminal
 - Beseitige **immer** Linksrekursion
 - Klammere gemeinsame Teilausdrücke nach links aus wo **möglich**
2. Erstelle Klasse für den Parser mit
 - **protected** Variable **currentToken**
 - Schnittstellenmethoden zum Scanner
 - **accept(t)** and **acceptIt()**
 - **public** Methode **parse**, welche ...
 - erstes Token via Scanner aus dem Eingabetext liest
 - die Parse-Methode des Start Nicht-Terminals S der CFG aufruft
 - Implementiere **protected** Parsing-Methoden
 - Methode **parseN** für jedes Nicht-Terminalsymbol **N**

3 Laufzeitorganisation

3.1 Einleitung

Die **Laufzeitorganisation** ist ein zentraler Aspekt des Compilerbaus und beschreibt, wie abstrakte Konzepte einer Hochsprache auf einer Zielmaschine umgesetzt werden. Sie bildet eine Brücke zwischen der Quellsprache und der zugrunde liegenden Hardware, indem sie sicherstellt, dass Programme korrekt ausgeführt werden können

3.1.1 Wesentliche Aufgaben der Laufzeitorganisation

1. Darstellung von Daten und Werten

Jeder Datentyp der Quellsprache (z.B. Ganzzahlen, Fließkommazahlen, Strings) muss auf der Zielmaschine repräsentiert werden. Dies umfasst:

- Die Auswahl geeigneter Speicherformate (z.B. Big-Endian vs. Little-Endian, feste oder variable Längen).
- Die Kodierung von Sonderfällen (z.B. Nullzeiger, NaN-Werte).

2. Auswertung von Ausdrücken und Handhabung von Zwischenergebnissen

- Mechanismen zur temporären Speicherung von Werten in Registern oder im Speicher.
- Behandlung von Operatoren und Operatorpräzedenz.

3. Speicherverwaltung

- **Globaler Speicher:** Für globale Variablen und Konstanten.
- **Lokaler Speicher:** Für Variablen, die innerhalb von Funktionen oder Prozeduren verwendet werden.
- **Heap:** Dynamisch allozierter Speicher (z.B. für Objekte oder Datenstrukturen).
- Unterstützung für Speicherbereinigung (z.B. Garbage Collection).

4. Routinen zur Prozedur- und Funktionsimplementierung

- Verwaltung von **Funktionsaufrufen:** Übergabe von Parametern und Rückgabewerten
- Aufbau und Verwaltung von **Call Stacks:**
 - Speicherung von Rücksprungadressen, lokalen Variablen und temporären Werten
- Unterstützung für Rekursion

5. Erweiterung für objektorientierten Sprachen

- Verwaltung von **Objekten und Methoden:**
 - Speicherung von Objekten auf dem Heap
 - Implementierung von **Methodentabellen** (z.B. für dynamisches Binden von Methoden)
- Unterstützung für **Vererbung** und **Polymorphismus**

3.1.2 Zielmaschinensupport

Die Zielmaschine (Hardware oder virtuelle Maschine) muss Mechanismen bereitstellen, um diese Aufgabe effizient umzusetzen:

- **Register:** Temporäre Speicherung und Berechnungen
- **Speicher:** Speicherung von Datenstrukturen und Programmezuständen
- **Maschineninstruktionen:** Unterstützung für Arithmetik, logische Operationen, Speicherzugriffe und Kontrollfluss
- **Stack:** Verwaltung von Funktionsaufrufen und -rückgaben sowie lokaler Variablen

3.1.3 Herausforderung des „Semantic Gap“

Der **semantische Spalt** beschreibt die Lücke zwischen den Konzepten der Hochsprache und den Fähigkeiten der Zielmaschine. Die Laufzeitorganisation versucht, diese Lücke zu schließen, indem sie komplexe Sprachmerkmale wie Arrays, Objekte und Prozeduren in die Grundoperationen der Zielmaschine übersetzt.

3.2 Triangle Abstract Machine

Die **Triangle Abstract Machine (TAM)** ist eine abstrakte Maschine, die entworfen wurde, um Programme aus der Triangle-Programmiersprache auszuführen. Sie ist ein essentielles Werkzeug zur Vermittlung von Konzepten im Compilerbau, da sie eine klare Trennung zwischen Maschinenressourcen und deren Verwendung demonstriert.

3.2.1 Speicherorganisation der TAM

Die TAM folgt einer **Harvard-Architektur** mit zwei getrennten Speicherbereichen:

1. **Instruktionsspeicher** (32-Bit-Worte): Speichert Programme und enthält Instruktionen.
2. **Datenspeicher** (16-Bit-Worte): Speichert Daten, wie Variablen, den Stack, den Heap und andere temporäre Werte.

Adressierung des Instruktionsspeichers

- **Code Base (CB):** Zeiger auf den Anfang des Programmcodes (konstant)
- **Code Top (CT):** Zeiger auf das Ende des Programmcodes (konstant)
- **Code Pointer (CP):** Zeiger auf die nächste auszuführende Instruktion (variabel)

Adressierung des Datenspeichers

- **Stack Base (SB):** Anfang des Stacks (konstant)
- **Stack Top (ST):** Zeiger auf die Spitze des Stacks (variabel)
- **Heap Base (HB):** Anfang des Heaps (konstant)
- **Heap Top (HT):** Zeiger auf das Ende des Heaps (variabel)
- **Heap Free (HP):** Zeiger auf den nächsten verfügbaren Speicherplatz im Heap (variabel)

Die Speicherbereiche sind klar getrennt, wobei **der Stack von unten nach oben wächst** und **der Heap von oben nach unten**. Nicht genutzter Speicher liegt zwischen diesen Bereichen.

3.2.2 TAM Speicherstruktur

Die Speicherbereiche sind logisch wie folgt organisiert:

1. **Code Store:** Beinhaltet Instuktionen und eventuell Intrinsics (primitive Operationen)
2. **Data Store:** Beinhaltet globale Variablen, Stack-Frames und den Heap

Memory Layout:

CB ... CT: Code Store

SB ... ST: Stack Segment (wächst nach oben)

HT ... HB: Heap Segment (wächst nach unten)

3.3 Auswertung von Ausdrücken

Die **Auswertung von Ausdrücken** in Compilerbau und Maschinenarchitektur zeigt, wie Hochsprachenoperationen auf Maschinenebene verarbeitet werden. Der Ansatz unterscheidet sich erheblich zwischen **Stack-Maschinen** und **Register-Maschinen**

3.3.1 Stack-Maschine

Eine Stack-Maschine verwendet einen **Stack**, um Operanden und Zwischenergebnisse zu verwalten. Dies ermöglicht eine einfache und klare Abarbeitung von Ausdrücken.

Beispiel: Auswertung von $d := a * a + 2 * a * b - 4 * a * c$

```
LOAD a ; Push a
LOAD a ; Push a
MUL ; a * a
LOADL 2 ; Push 2
LOAD a ; Push a
MUL ; 2 * a
LOAD b ; Push b
MUL ; 2 * a * b
ADD ; a * a + 2 * a * b
LOADL 4 ; Push 4
LOAD a ; Push a
MUL ; 4 * a
LOAD c ; Push c
MUL ; 4 * a * c
SUB ; (a * a + 2 * a * b) - (4 * a * c)
STORE d ; Store result in d
```

Ablauf (Stack-Zustand):

1. LOAD a -> Stack: [a]
2. LOAD a -> Stack: [a, a]
3. MUL -> Stack: [a * a]
4. LOADL 2 -> Stack: [a * a, 2]
5. LOAD a -> Stack: [a * a, 2, a]
6. MUL -> Stack: [a * a, 2 * a]
7. LOAD b -> Stack: [a * a, 2 * a, b]

8. MUL -> Stack: $[a * a, 2 * a * b]$

9. ...

3.3.2 Register-Maschine

Eine Register-Maschine verwendet **Register** zur Speicherung von Operanden und Zwischenergebnissen. Dies erfordert eine präzise Verwaltung der begrenzten Anzahl von Registern

Typische Eigenschaften

- Operanden und Zwischenergebnisse werden in Registern gehalten
- Instruktionen spezifizieren die Register direkt
- Zwischenergebnisse müssen ggf. in den Speicher ausgelagert werden, wenn die Registeranzahl nicht ausreicht

Vorteile

- Effizientere Nutzung der Hardware, da Registerzugriffe schneller sind als Speicherzugriffe
- Weniger Speicherzugriffe als bei einer Stack-Maschine

Herausforderungen

- Register-Allokation: Compiler muss Zwischenergebnisse auf Register verteilen
- Bei zu komplexen Ausdrücken müssen Register-Inhalte in den Speicher ausgelagert werden

Beispiel: Auswertung von $d := a * a + 2 * a * b - 4 * a * c$

Register-Maschinen-Code:

```
LOAD R1 a ; R1 = a
MULT R1 a ; R1 = a * a
LOAD R2 2 ; R2 = 2
MULT R2 a ; R2 = 2 * a
MULT R2 b ; R2 = 2 * a * b
ADD R1 R2 ; R1 = a * a + 2 * a * b
LOAD R2 4 ; R2 = 4
MULT R2 a ; R2 = 4 * a
MULT R2 c ; R2 = 4 * a * c
SUB R1 R2 ; R1 = a * a + 2 * a * b - 4 * a * c
STORE R1 d ; d = R1
```